Predicting Residential Sale Prices in Cook County: Benchmarking GUIDE Trees against CART and Ensembles

Arundhati Singh
STAT 443 - Spring 2026

Abstract

This project predicts residential property sale prices in Cook County using a dataset of 98,714 records. The data, originally used by the Cook County Assessor's Computer Assisted Mass Appraisal system, contains every arm's-length residential sale and includes 83 columns of both qualitative and quantitative features. The richness of the dataset provides an excellent opportunity to benchmark regression tree algorithms by comparing not only their predictive performance but also the variable importance scores they produce. The interpretability of GUIDE's trees also provides valuable insight into which features truly drive predictions.

Prior to modeling, a thorough exploratory data analysis was conducted to reduce redundancy and skewness. Missing values were also examined and classified into MCAR, MAR, and MNAR categories (Section 1.3). Section 1.3 reveals a huge degree of structural missingness by grouping features according to missing percentage and the median sale price with versus without the feature (see Section 3.6 for suggested improvement). The large number of identical missing rates in Table 1 and the notable sparsity of features within each modeling group further indicate structural missingness due to property-type-specific (uncommon) features. Although standard CART algorithms frequently select continuous variables as the primary split points (because they offer far more candidate split values), this does not imply they are the most important predictors. This well-known selection bias is clearly visible in the variable importance scores shown in Figure 12 (rpart), Figure 13 (sklearn DecisionTreeRegressor), Figure 14 (Random Forest), and Figure 15 (LightGBM), all of which ranked latitude as the top predictor (with the top four features continuously rotating among themselves across the four plots) simply because they have a large number of distinct values. The results section (Section 3) and the GUIDE trees (Section 2.4) show the importance scores provided by GUIDE, indicating that GUIDE directly addressed the selection bias through an unbiased variable selection procedure, thus providing unbiased importance scores. In addition to GUIDE, the project implements and compares **linear discriminant analysis** (Section 2.2), **linear and ridge regression** (Section 2.3), **decision tree regressors** (Sections 2.5 and 2.6), **random forests** (Section 2.7), and the **gradient-boosted LightGBM** model (Section 2.8).

As per [Stat 443: Slide 133], GUIDE first calculates the Chi-squared ranking and, in cases of a tie, utilizes the Hilferty score for breaking ties. We implemented this procedure using two engineered targets - "tertiles of log(Sale Price) and the sign of residuals calculated from the median of log(Sale Price) [Stat 443: Slide 338]". "Pure Market Filter" ranked first under the tertile target but last under the residual-based target (Section 1.2). This anomaly led us to investigate the distribution of "Sale Price" within the two subgroups of "Pure Market Filter" and to assess the other top-ranked binary variables (Section 1.2.1), which revealed that only Pure Market Filter has substantially different distributions.

Next, we scanned the columns for features that are mathematically derivable from a base feature and dropped them to reduce redundancy and retain pure signal (Section 1.2.2). In addition,

we dropped other redundant columns that are either verbose or simply do not provide any signal (Section 1.2.3). We also ranked the continuous variables by their skewness and chose to log-transform the top-3 skewed continuous variables so that they approach a closer-to-normal distribution (Section 1.2.4). This helps reduce the disproportionate influence of outliers on error metrics such as RMSE and MSE.

Next, we assessed which error metrics would be most suited for validating the forecasted regression values by examining the distribution of residuals and concluded that mean-squared error was appropriate given their approximately normal shape (Section 1.2.6). We also analyzed whether a year-based train-test split would be appropriate and ultimately retained sales from 2013–2017 for training and 2018–2019 for validation after observing steady price growth (Section 1.2.5).

Next, we performed basic Exploratory Data Analysis to understand the structure of the data better before modeling it with Classification and Regression Trees. We computed the Pearson Correlation Coefficient between pairs of numerical features and filtered out pairs with a high correlation factor of 0.8 and above (Section 1.4.1). We then filtered out pairs sharing similar proportions of missingness. The variable to be removed from each pair was decided by cross-referencing with GUIDE importance scores. Dropping these redundant features had virtually no impact on Ridge regression performance (Section 2.3), highlighting its inability to capture non-linear interactions.

Aside from Pearson correlations, we used two other feature-ranking approaches that target categorical predictors. ANOVA's F-statistic ranks each categorical variable by how well its category means separate continuous Sale Price. The Wilson-Hilferty approach approximates GUIDE's root-node chi-squared scoring. Section 1.4.2 and Section 1.4.3 assess the distribution of the top-3 ranked features returned by ANOVA and Wilson-Hilferty (Chi-squared) against the log of sale prices. We also evaluated the quality of these rankings via boxplots (Section 1.4.6), again giving a clear comparison between ANOVA and the Chi-squared + Hilferty test used by GUIDE. This implicitly draws a comparison between rpart and GUIDE.

Finally, we moved onto modeling the feature-engineered dataset. Linear discriminant analysis was first applied after binning the log sale price into three quartiles; one-hot encoding of categorical variables improved cross-validation accuracy from 0.543 to 0.588. A comparison of the LDA projections before and after one-hot encoding (Section 2.2) shows that the separation between the three tertiles is much clearer in the second plot. We then fitted GUIDE regression trees using both multiple-linear and stepwise-linear leaf models (Section 2.4). A temporal hold-out evaluation (training on 2013–2017 sales and testing on 2018–2019, $n = 23{,}810$) was adopted after it proved superior to 10-fold cross-validation. On the hold-out set, stepwise-linear leaves achieved the best performance with Log MSE = 0.2315, Log RMSE = 0.4811, and Log R^2 = 0.6487, narrowly outperforming multiple-linear leaves. Feature engineering comparisons further showed that raw square footage variables (v3) substantially outperformed price-per-square-foot features (v2), due to leakage of the sale price from the training dataset into the test set via price-per-square-foot, thus resulting in biased forecasts.

Linear and ridge regression achieved identical performance (Log RMSE ≈ 0.595, Log $R^2 \approx 0.463$) (Section 2.3), showing minimal benefit from penalizing misforecasts and highlighting linear models' inability to capture non-linear interactions. Among the single regression trees, GUIDE's regression tree performed best (Log $R^2 = 0.6487$) (Section 2.4), substantially outperforming classical CART models (rpart (Section 2.5) and DecisionTreeRegressor (Section 2.6), Log $R^2 \approx 0.50$–0.52), both of which ranked latitude highest due to its large number of distinct values. Among ensembles, LightGBM (Section 2.8) delivered the lowest RMSE while GUIDE Random Forest achieved the highest log-scale R^2 of 0.7938 (Section 3).

1 Data

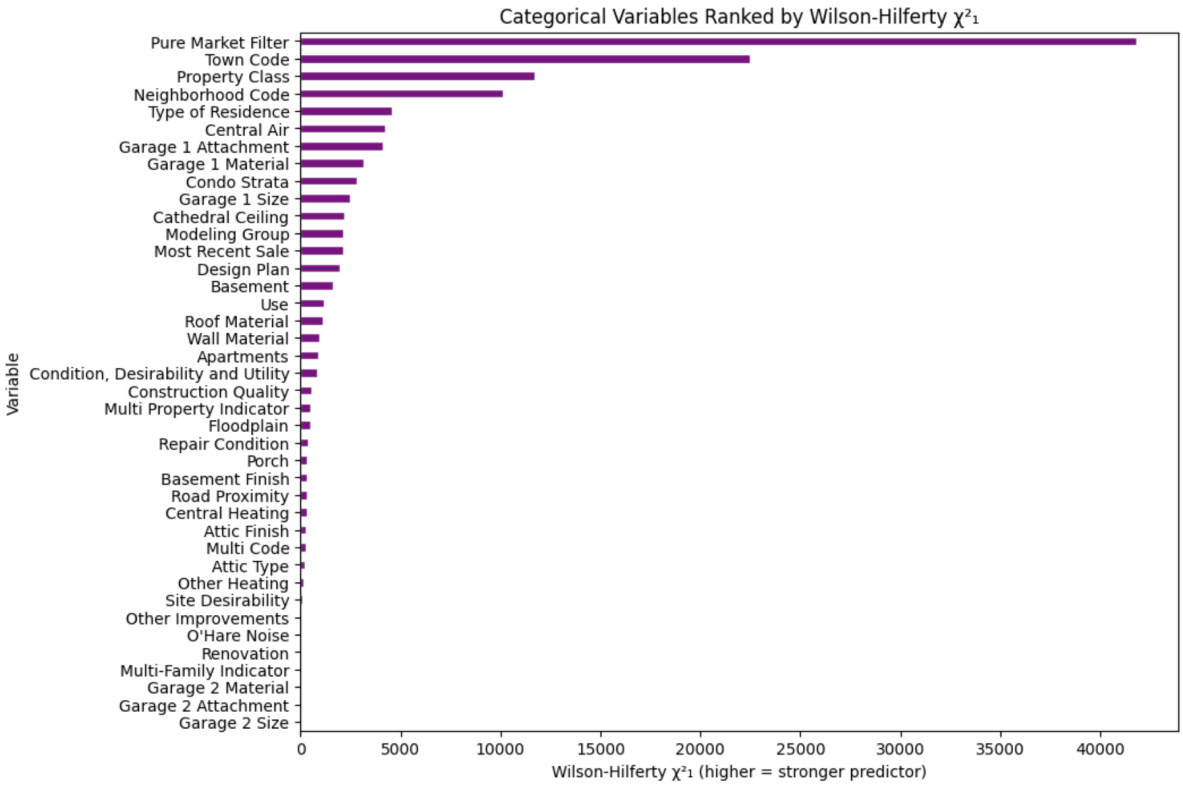
1.1 Source

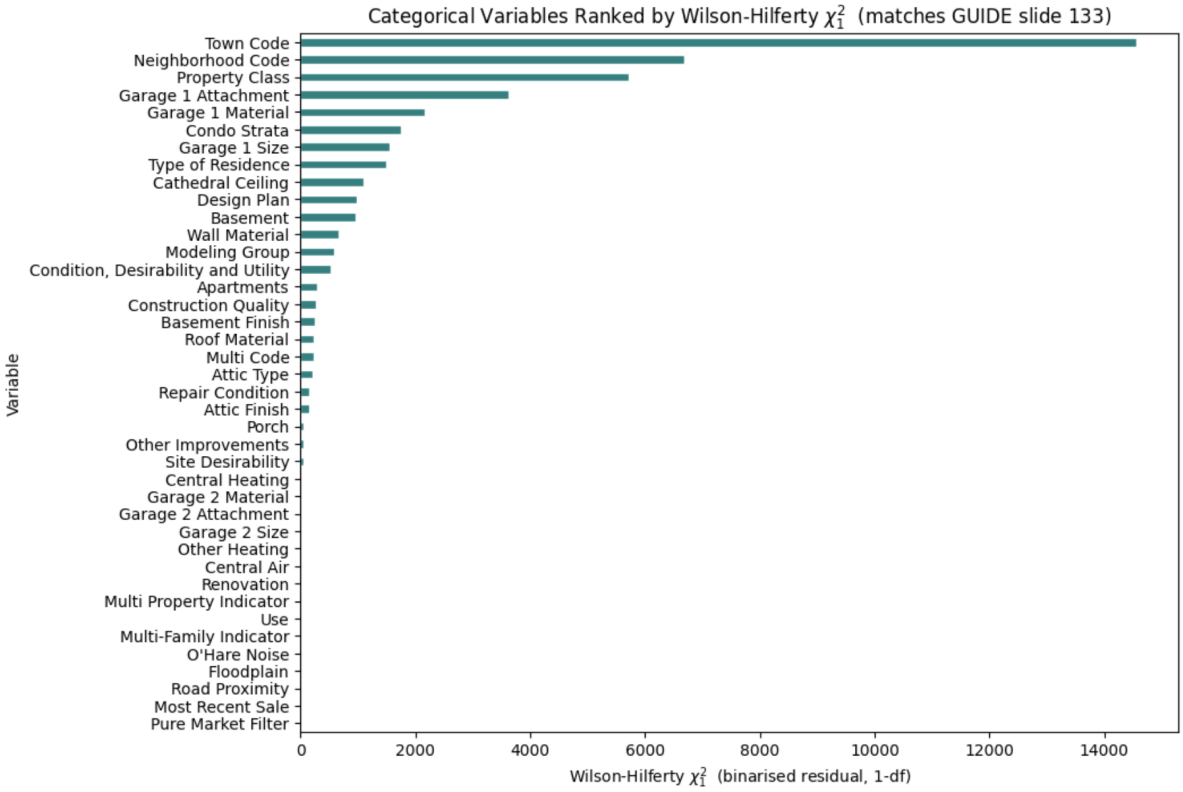
The data are taken from the Cook County Government Open Data Portal:
`https://datacatalog.cookcountyil.gov/Property-Taxation/Assessor-Archived-05-11-2022-Residentia`
`5pge-nu6u/about_data`

As mentioned above, this is the data set of residential sales used by the Cook County Assessor in their Computer Assisted Mass Appraisal system used to assess residential property values.

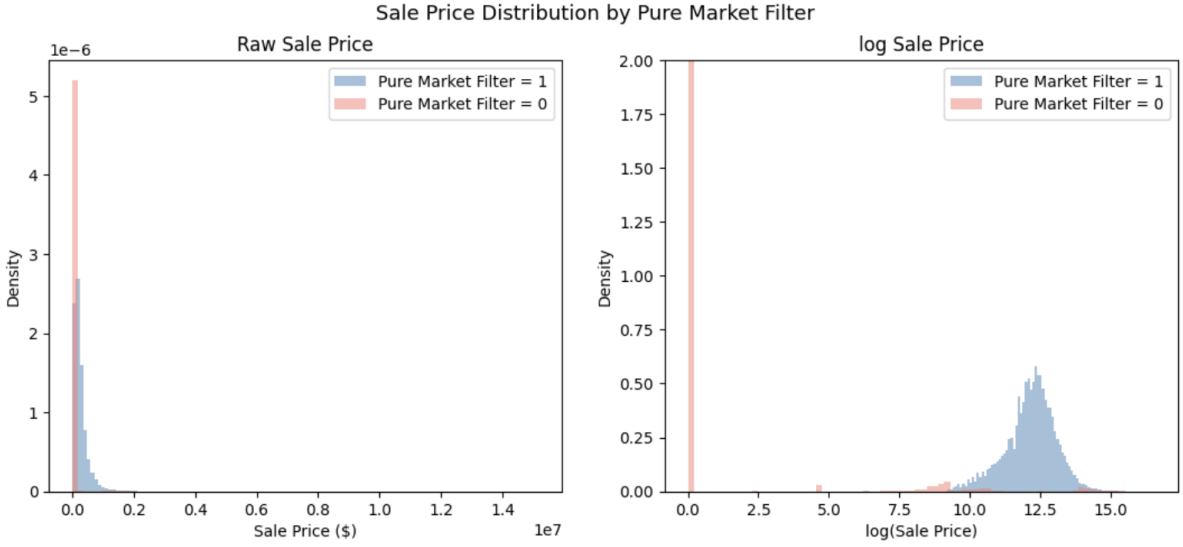
1.2 Cleaning, Filtering, and Feature Engineering

- Originally, there are 98,714 rows with 83 columns. Post cleaning, filtering, and feature engineering, the number of rows and columns change.
- To explore all the high impact variables, we approximate GUIDE's variable selection methodology by using the Chi-squared test + Hilferty. First, we bin the log of sale price into three categories (tertiles) and treat that as the target variable. In the second image, we treat the negative/positive classification of the residuals of log of sales as the target variable.





While doing so produces different rankings, the most significant mismatch observed was that of "Pure Market Filter". While the first figure ranks "Pure Market Filter" as first, the second one ranks it at the bottom. Given this result, we derived the raw dollar distribution and log sale price distribution of residences that do and do not satisfy the "Pure Market Filter".



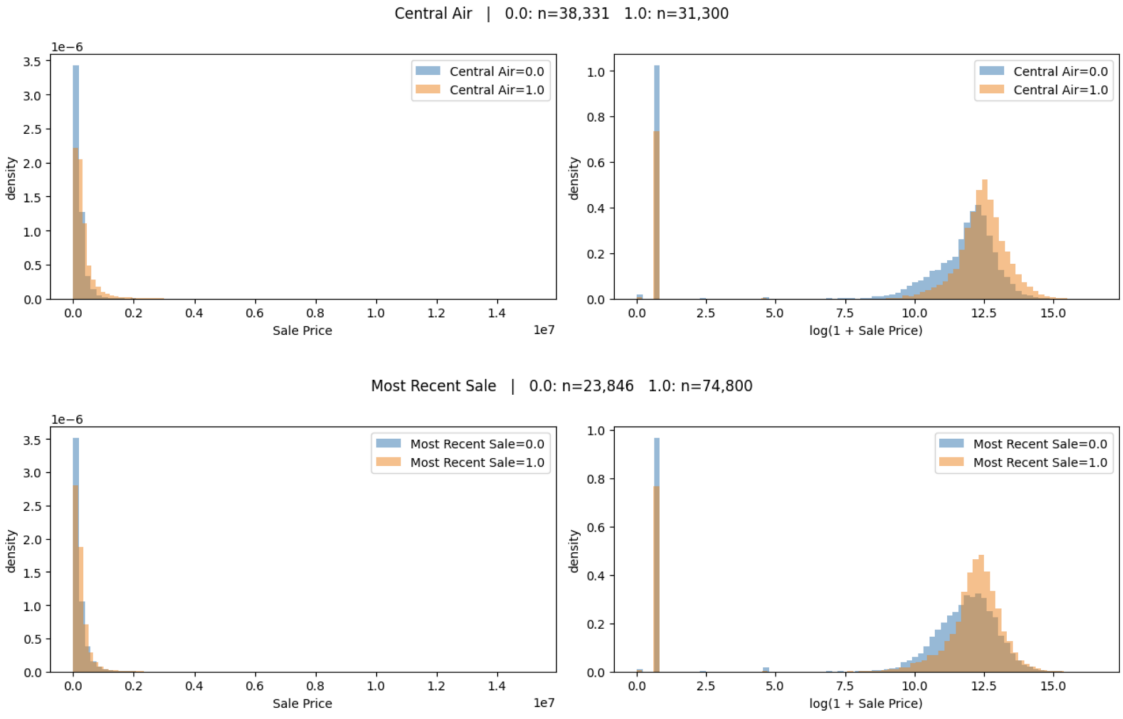
- The selling prices of properties with and without the "Pure Market Filter" follow very different price distributions, which can potentially distort the accuracy of regression predictions. The raw dollar distributions and the log distributions followed by transactions that do and don't satisfy the "Pure Market Filter" are included below for reference.

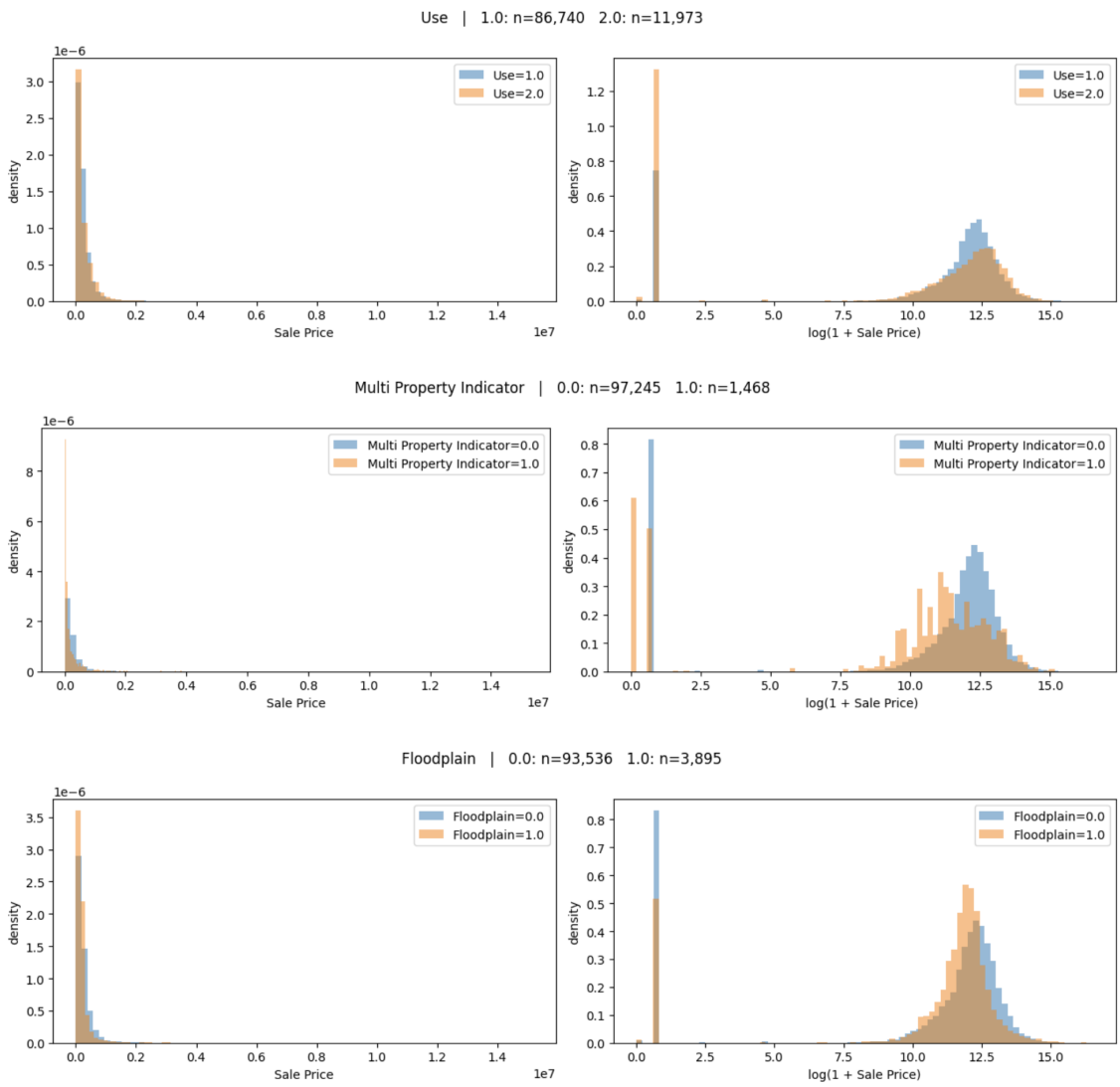
```
Pure Market Filter = 1 (market sales):
  Count: 80,743
  Mean:   $272,778
  Median: $205,000
  Min:    $10,005
  Max:    $9,400,000

Pure Market Filter = 0 (non-market sales):
  Count: 17,970
  Mean:   $26,695
  Median: $1
  Min:    $0
  Max:    $15,177,429

% of non-market sales under $1,000: 93.7%
% of market sales under $1,000:     0.0%
```

- There are 80,743 transactions that satisfy the pure market filter with a mean value of $272,778, a median value of $205,000, a min value of $10,000, and a max value of $9,400,000. These statistics seem plausible for arm's-length residential sales in Cook County. Meanwhile, this isn't the case for transactions that do not satisfy the pure market filter.

1.2.1 Binary-flag audit beyond Pure Market Filter

After noticing completely different distributions between the two subcategories of "Pure Market Filter", we filtered out all the binary categories within the top 25 ranks of the Chi-squared test + Hilferty to assess for the same anomaly in distribution. However, the distribution of log of sale price and raw sale price did not exhibit the same degree of separation between the subcategories of every other binary categorical variable, unlike Pure Market Filter.





1.2.2 Removing derived (mathematically redundant) columns

- To reduce redundancy, we remove all the columns that can be derived from other base columns in the dataset. Based on all the column descriptions, the derived columns are as follows:
  - **Age Squared**, **Age Decade**, and **Age Decade Squared** can be derived mathematically from **Age**.
  - **Lot Size Squared** and **Square root of lot size** can be derived from **Land Square Feet**.
  - **Improvement Size Squared** and **Square root of improvement size** can be derived from **Building Square Feet**.
  - **Square root of age** can be derived from **Age**.

- **Town and Neighborhood** is a string concatenation of **Town Code** and **Neighborhood Code**.

- **Neigborhood Code (mapping)** is a verbatim copy of **Neighborhood Code**.

- **Sale Quarter**, **Sale Half-Year**, **Sale Half of Year**, **Sale Quarter of Year**, and **Sale Month of Year** are all derived from **Sale Date**. We retain **Sale Date** and **Sale Year** and drop the rest.

- **Garage Indicator** is derived from **Garage 1 Size** (1 if a garage exists, 0 otherwise).

Each of the columns to be dropped was first backtracked to verify that it could indeed be derived from the base columns. After this step, the dataset has 80,743 rows and 66 columns.

1.2.3 Dropping pure identifiers and circular features

- We then drop three pure identifiers that carry no predictive signal once geography has been encoded elsewhere:

  - **PIN**: Unique Permanent Identification Number for each property. All PINs are 14 digits: 2 digits for area + 2 digits for sub area + 2 digits for block + 2 digits for parcel + 4 digits for the multicode (according to the definition stated by the website). Even though PIN encodes geographic information, this is already captured by 'Neighborhood Code', 'Town Code', and 'Census Tract'. Since the PIN is unique to each property in the dataset, it is very hard for any model, including GUIDE, Random Forest, or Gradient Boosting models, to generalize on it. At the same time, we are not losing any geographic or location-specific information that can be used by these models, since it is already encoded in other features.

  - **Deed No.**: A unique deed number assigned to each sale transaction (according to the definition stated by the website). This is a record-keeping identifier with no relationship to property value.

  - **Property Address**: Property street address, not the address of the taxpayer (according to the definition stated by the website). Location information is already captured by other geographical features like **Neighborhood Code**, **Town Code**, **Census Tract**, **Longitude**, and **Latitude**. The content within the "Property Address" column is very hard to generalize on given the verbosity of the content in each row. At the same time, we do not lose any geographical information since it is already encoded in other columns.

  After dropping these three columns the dataset has 80,743 rows and 63 columns.

- We also drop the Cook County Assessor's own predictions – **Estimate (Land)** and **Estimate (Building)**, defined by the column descriptions as the "final estimated market value of land from the prior tax year" and "final estimated market value of building from the prior tax year" respectively. Including the Board of Review's own predictions would incorrectly distort the predictions since it would rank high in terms of features that can be used for predicting the sale price of apartments, thus taking off the attention from other geographical, physical,

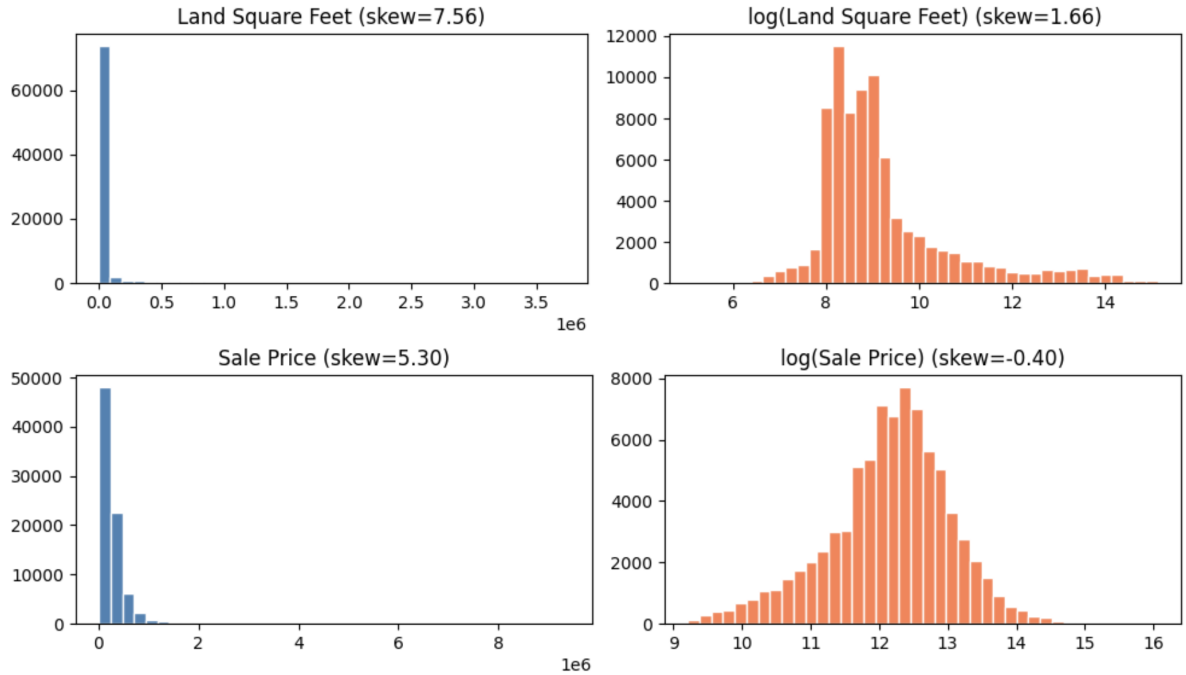
or qualitative predictors. After removing these 2 columns, the dataset has 80,743 rows and 61 columns.

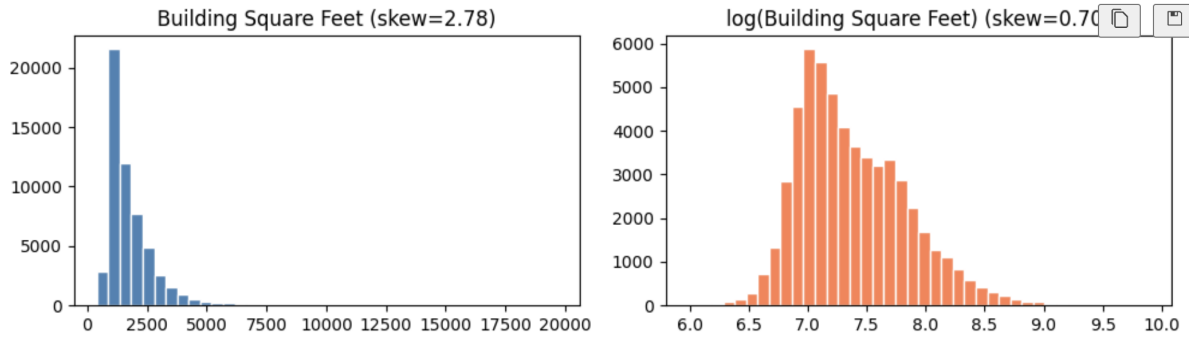
1.2.4 Skewness ranking and log-transform decision

- We identified the top few continuous features that are heavily skewed by calling the "skew" function on these continuous columns. We obtained the following results after doing so:

```
Skewness for continuous columns:
Land Square Feet          7.556093
Sale Price                5.296046
Rooms                     3.566931
Bedrooms                  3.345441
Apartments                2.966069
Building Square Feet      2.775936
Full Baths                2.116920
Fireplaces                1.521311
Half Baths                1.111558
Age                       0.309562
Latitude                 -0.506481
Longitude                -1.068875
```

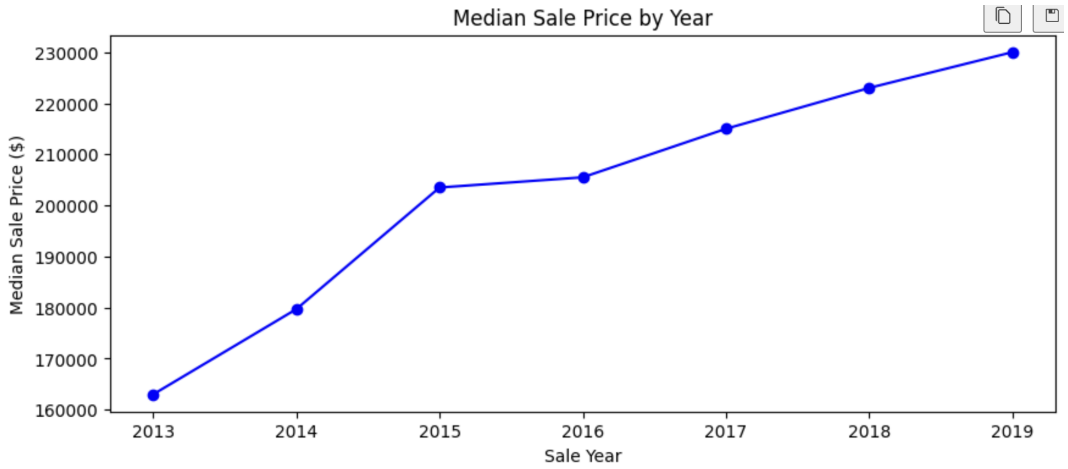
- We log-transform the top-3 skewed continuous features - **Land Square Feet**, **Sale Price**, and **Building Square Feet**. The plots of these columns before and after the transformation are attached below.

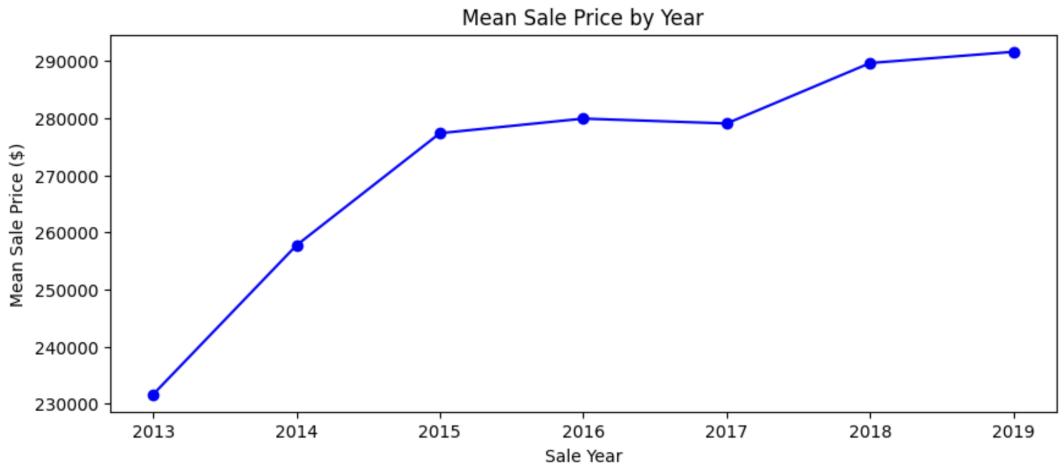




1.2.5 Train/test split: temporal hold-out vs cross-validation

- Since the dataset spans multiple years (2013 to 2019), we compared two approaches for measuring the accuracy of the predictions made by GUIDE. One way to validate the results is using **cross-validation**. The other way is using **temporal hold-out validation**, wherein we train the model on transactions from 2013-2017 and evaluate it on transactions from 2018-2019. To determine which approach best fits our use case, we examined the trend in sale prices from 2013 to 2019 as is shown in the figures below:



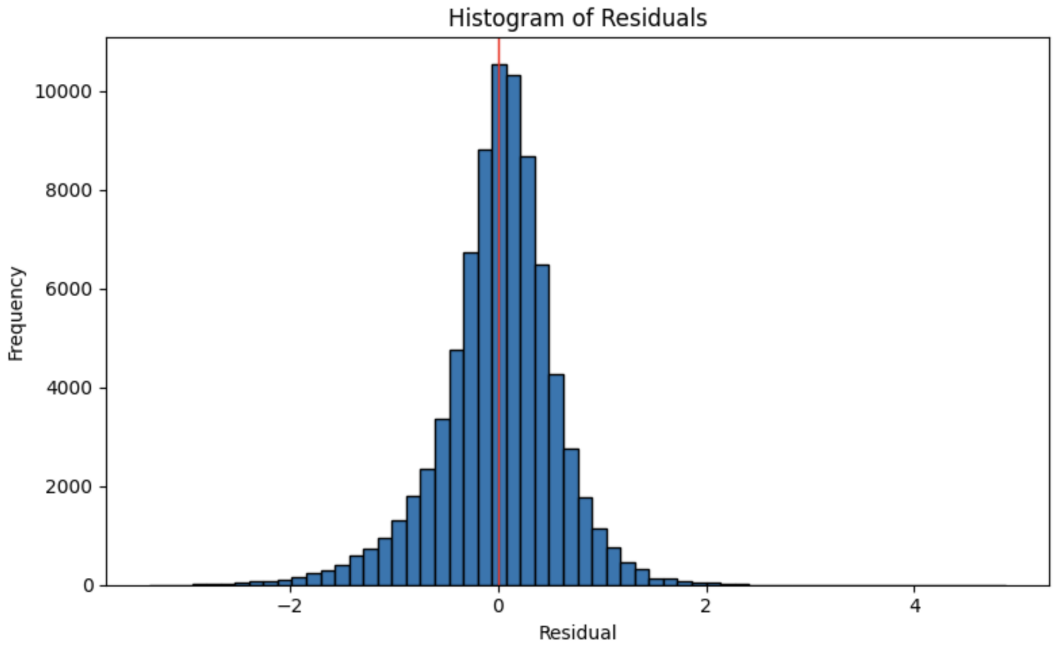


Since the sale price is right-skewed, as can be seen in the images above, we decided to closely

follow the trend displayed by "Median Sale Price by Year". The trend is most increasing with an intermittent plateau from 2015–2016, indicating that time also plays a factor in the pricing of residences, thus indicating that a temporal hold-out might be a better option than cross-validation.

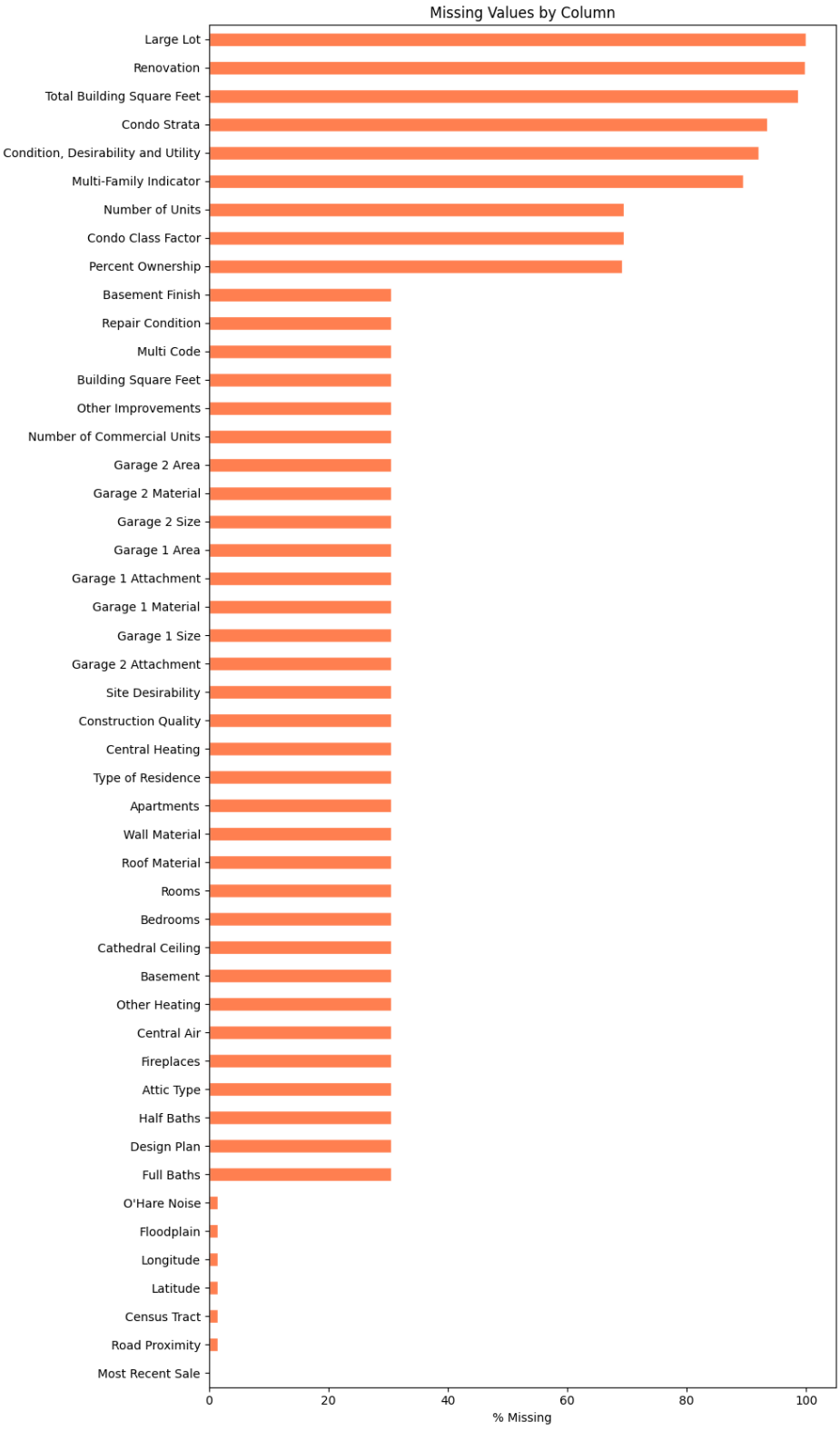
1.2.6 Choice of error metric: mean-squared vs median-squared

To understand whether it would be best to choose mean-squared error or median-squared error when prompted by GUIDE, we plotted the distribution of residuals. Since the distribution is approximately normal, the mean is a reliable measure of central tendency, and we can safely use mean-squared error. Had the residuals been skewed, median-squared error would have been the more robust choice since in that scenario, we would want to reduce the influence of the outliers. Since we calculate root-mean-squared error and mean-squared error throughout the notebook, a skewed distribution of residuals would disproportionately inflate these metrics due to the outsized influence of outliers.



1.3 Missing Data Analysis (MCAR, MAR, MNAR)

Before diving into the structural missingness of the dataset, we first examine the overall proportion of missing data across all features.



To assess whether the missingness is structural rather than random, we compare the median sale price of records where each feature is present versus absent. A large difference in median sale price between the two groups suggests that the missingness is tied to property characteristics, indicating

MAR or MNAR rather than MCAR. The table below summarizes this comparison, sorted by the percentage of missing values.

Table 1: Missing Data Summary with Median Sale Price Comparison

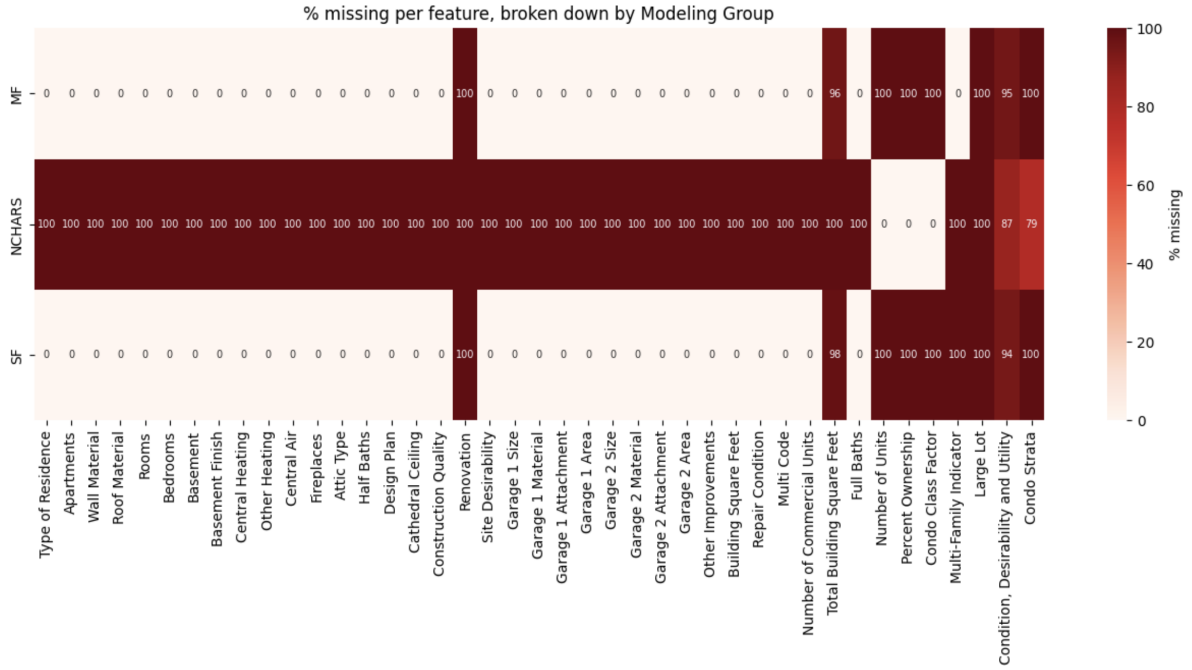
Column	**% Missing**	**Median (present)**	**Median (absent)**	**N present**	**N absent**
High Missingness (>89%)					
Large Lot	100.0%	—	$205,000	0	80,743
Renovation	99.9%	$550,000	$205,000	105	80,638
Total Building Square Feet	98.6%	$96,520	$205,000	1,116	79,627
Condo Strata	93.4%	$175,000	$208,000	5,308	75,435
Condition, Desirability and Utility	92.1%	$225,000	$202,000	6,373	74,370
Multi-Family Indicator	89.5%	$225,000	$202,000	8,498	72,245
Condo-Related Cluster (∼69%)					
Number of Units	69.4%	$181,500	$215,000	24,689	56,054
Condo Class Factor	69.4%	$181,500	$215,000	24,689	56,054
Percent Ownership	69.3%	$181,000	$215,000	24,825	55,918
Modeling Group Cluster (30.6%)					
Basement Finish	30.6%	$215,000	$181,500	56,053	24,690
Repair Condition	30.6%	$215,000	$181,500	56,054	24,689
Multi Code	30.6%	$215,000	$181,500	56,054	24,689
Building Square Feet	30.6%	$215,000	$181,500	56,054	24,689
Other Improvements	30.6%	$215,000	$181,500	56,054	24,689
Number of Commercial Units	30.6%	$215,000	$181,500	56,054	24,689
Garage 2 Area	30.6%	$215,000	$181,500	56,054	24,689
Garage 2 Material	30.6%	$215,000	$181,500	56,054	24,689
Garage 2 Size	30.6%	$215,000	$181,500	56,054	24,689
Garage 1 Area	30.6%	$215,000	$181,500	56,054	24,689
Garage 1 Attachment	30.6%	$215,000	$181,500	56,054	24,689
Garage 1 Material	30.6%	$215,000	$181,500	56,054	24,689
Garage 1 Size	30.6%	$215,000	$181,500	56,054	24,689
Garage 2 Attachment	30.6%	$215,000	$181,500	56,054	24,689
Site Desirability	30.6%	$215,000	$181,500	56,054	24,689
Construction Quality	30.6%	$215,000	$181,500	56,054	24,689
Central Heating	30.6%	$215,000	$181,500	56,054	24,689
Type of Residence	30.6%	$215,000	$181,500	56,054	24,689
Apartments	30.6%	$215,000	$181,500	56,054	24,689
Wall Material	30.6%	$215,000	$181,500	56,054	24,689
Roof Material	30.6%	$215,000	$181,500	56,054	24,689
Rooms	30.6%	$215,000	$181,500	56,054	24,689
Bedrooms	30.6%	$215,000	$181,500	56,054	24,689
Cathedral Ceiling	30.6%	$215,000	$181,500	56,054	24,689
Basement	30.6%	$215,000	$181,500	56,054	24,689
Other Heating	30.6%	$215,000	$181,500	56,054	24,689
Central Air	30.6%	$215,000	$181,500	56,054	24,689
Fireplaces	30.6%	$215,000	$181,500	56,054	24,689
Attic Type	30.6%	$215,000	$181,500	56,054	24,689
Half Baths	30.6%	$215,000	$181,500	56,054	24,689
Design Plan	30.6%	$215,000	$181,500	56,054	24,689
Full Baths	30.6%	$215,000	$181,500	56,054	24,689

Several patterns of "structural missingness" emerge from this table. A large block of features, from "Basement Finish" through "Full Baths", share an identical missing rate of 30.6% with identical counts of 56,054 present and 24,689 missing records. This strongly suggests that these features belong to a single modeling group and are collectively absent when a property falls outside that group. The median sale price for records with these features present ($215,000) versus absent ($181,500) further confirms that this missingness is structural rather than random, thus falling into MNAR.

At the other end, features like "Renovation" (99.9% missing, median $550,000 when present) and "Total Building Square Feet" (98.6% missing, median $96,520 when present) show extreme differences in median sale price, indicating that these features are only recorded for a very specific subset of properties.

Meanwhile, "Number of Units", "Condo Class Factor", and "Percent Ownership" cluster together at approximately 69.4% missing with nearly identical counts, suggesting another structural grouping, likely condominiums.

To confirm these structural groupings, we plot the percentage of missing data per feature broken down by modeling group.

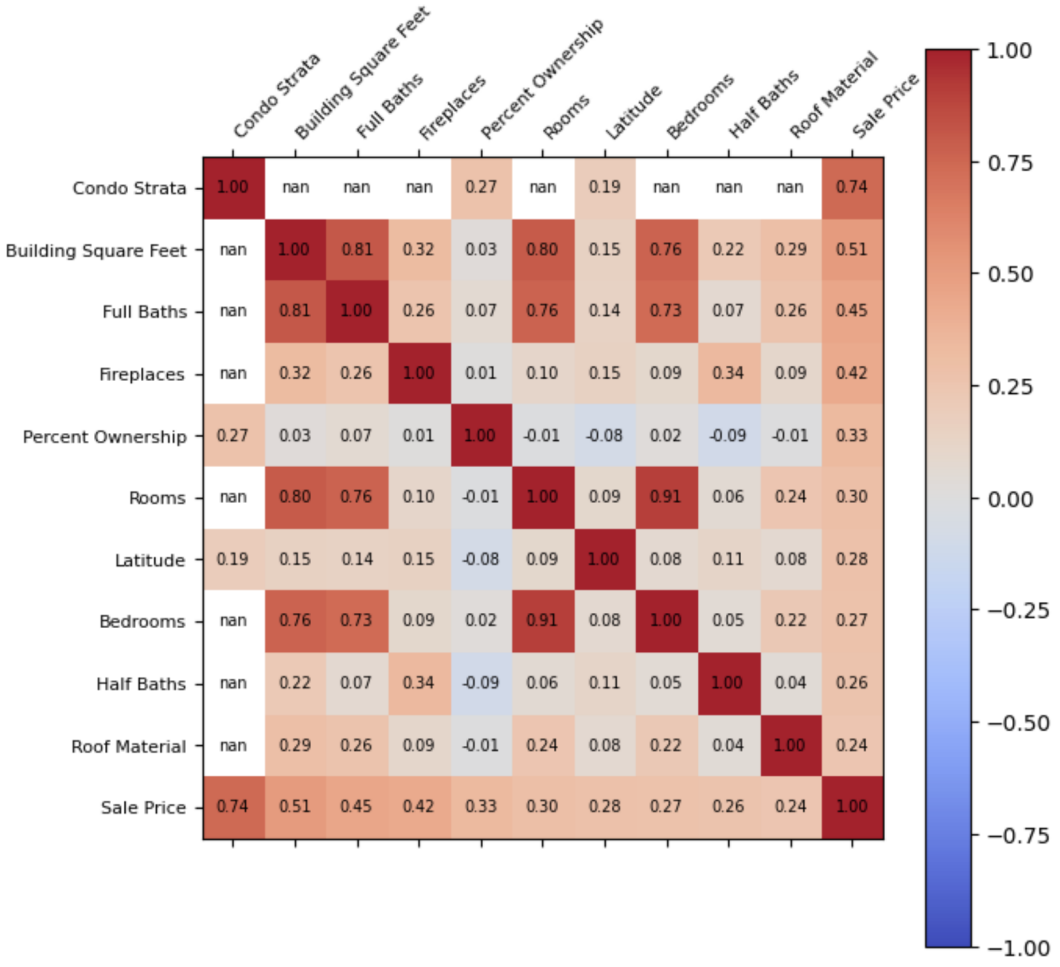


This heatmap confirms that the missingness is almost entirely determined by modeling group membership. Features that are 100% missing within a given modeling group are structurally absent, as they simply do not apply to that property type. Likewise, features that are exclusively present in a single modeling group carry no information for the remaining groups. This classifies the majority of the missing data as MNAR, since the missingness is not random and the reason for missingness is directly tied to the property type / modeling group itself.

1.4 Exploratory Data Analysis

1.4.1 Pairwise Pearson correlations and the $|r| > 0.8$ filter

- To understand the relationship of each feature with the target variable (`Sale Price`), we produced a correlation heatmap showing the pairwise Pearson correlation coefficients among all features. This heatmap helps identify both strong predictors (features highly correlated with Sale Price) and redundant features (pairs of predictors with very high correlation), allowing us to remove one variable from highly correlated pairs to reduce multicollinearity.



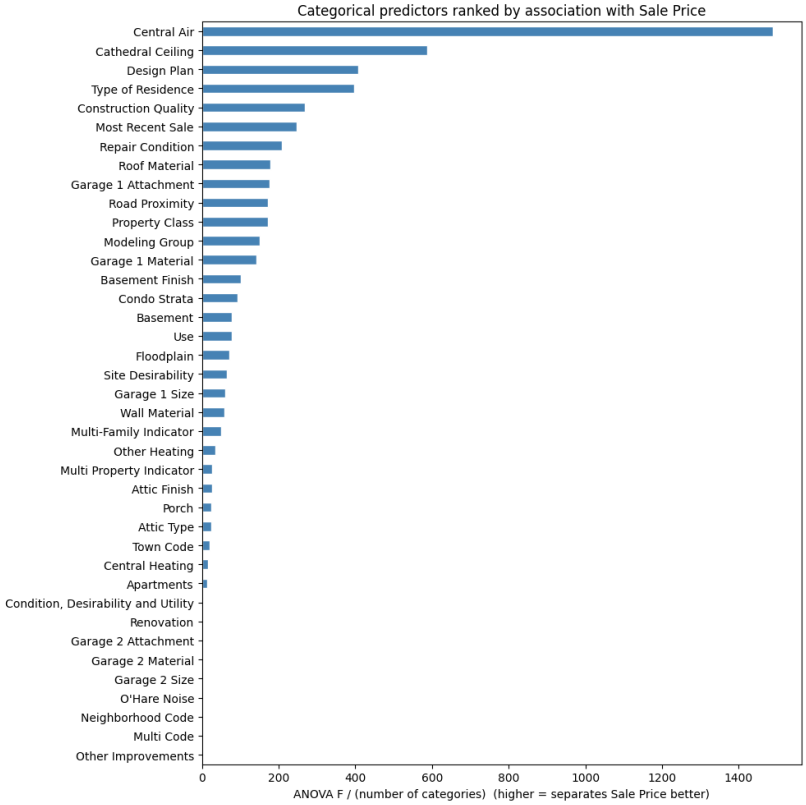
	Var 1	Var 2	r
0	Property Class	Multi-Family Indicator	1.000000
1	Apartments	Use	0.876483
2	Rooms	Bedrooms	0.905522
3	Rooms	Building Square Feet	0.802544
4	Renovation	Garage 2 Area	1.000000
5	Garage 1 Attachment	Garage 1 Area	0.854652
6	Garage 2 Size	Garage 2 Material	-0.905621
7	Garage 2 Size	Garage 2 Attachment	-0.909876
8	Garage 2 Material	Garage 2 Attachment	0.920190
9	Building Square Feet	Full Baths	0.809094
10	Number of Commercial Units	Multi-Family Indicator	0.895663

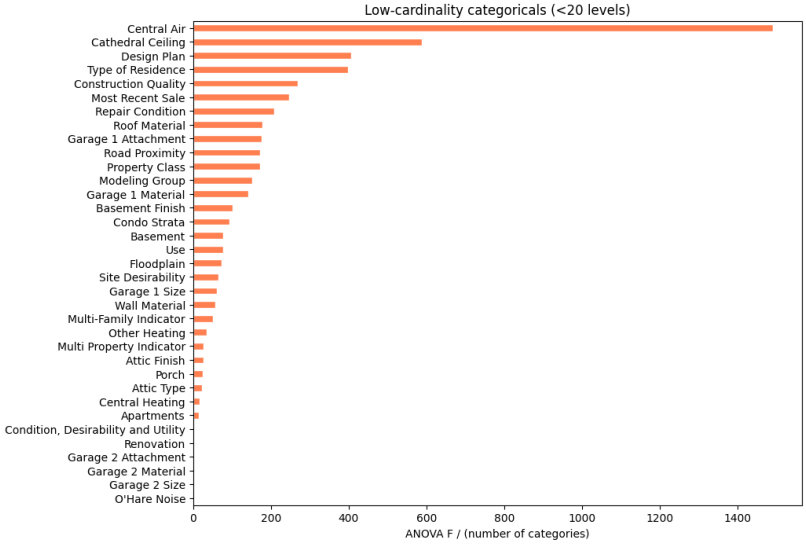
Figure 1: Highly correlated predictor pairs ($|r| > 0.8$)

	Variable A	Variable B	Correlation	Missing A (%)	Missing B (%)	Diff in Missing (%)
0	Property Class	Multi-Family Indicator	1.0000	0.00	89.45	89.45
11	Renovation	Garage 2 Area	1.0000	99.87	30.33	69.54
17	Multi Code	Total Building Square Feet	0.7992	30.33	98.49	68.16
18	Number of Commercial Units	Multi-Family Indicator	0.9010	30.33	89.45	59.12
4	Apartments	Use	0.8762	30.33	0.00	30.33
10	Attic Type	Attic Finish	0.7964	30.33	0.00	30.33
1	Town Code	Census Tract	0.7444	0.00	1.74	1.74
5	Rooms	Bedrooms	0.9062	30.33	30.33	0.00
6	Rooms	log Building Square Feet	0.7494	30.33	30.33	0.00
7	Rooms	Full Baths	0.7641	30.33	30.33	0.00
8	Bedrooms	log Building Square Feet	0.7222	30.33	30.33	0.00
3	Apartments	Bedrooms	0.7077	30.33	30.33	0.00
12	Garage 1 Attachment	Garage 1 Area	0.8532	30.33	30.33	0.00
13	Garage 2 Size	Garage 2 Material	0.8908	30.33	30.33	0.00
14	Garage 2 Size	Garage 2 Attachment	0.9063	30.33	30.33	0.00
15	Garage 2 Material	Garage 2 Attachment	0.9190	30.33	30.33	0.00
16	log Building Square Feet	Full Baths	0.7800	30.33	30.33	0.00
2	Apartments	Rooms	0.7597	30.33	30.33	0.00
9	Bedrooms	Full Baths	0.7292	30.33	30.33	0.00

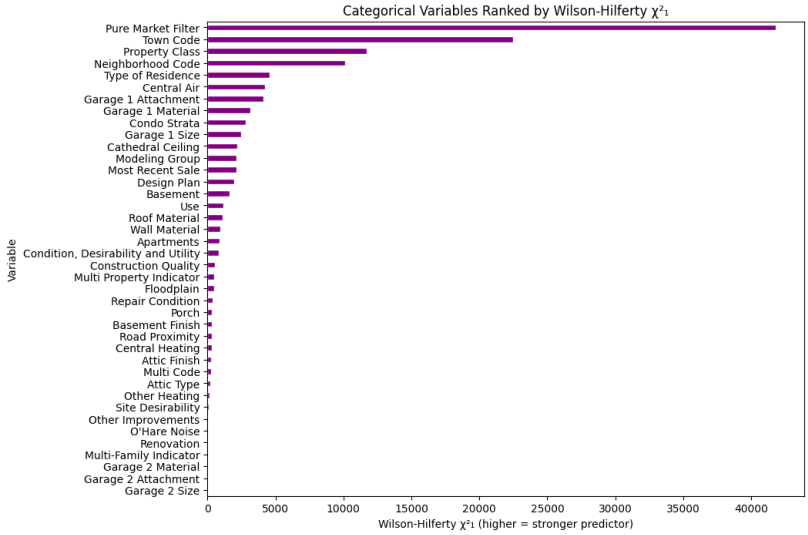
From the table above, we observe two distinct groups of correlated pairs. The first group (rows 0, 11, 17, 18, 4) exhibits a large difference in missing percentages between the two variables, reinforcing the structural missingness patterns identified earlier; these pairs are correlated but belong to different modeling groups. The second group (rows 5–9, 12–16, 2, 3) shows a 0% difference in missing values, meaning both variables in each pair share the same missingness pattern. Since these pairs are both highly correlated and missing in lockstep, retaining both would introduce redundancy. To decide which variable to drop from each pair, we cross-reference with the importance scores returned by GUIDE and retain the variable with the higher importance score.

1.4.2 Categorical-predictor ranking: ANOVA F-statistic

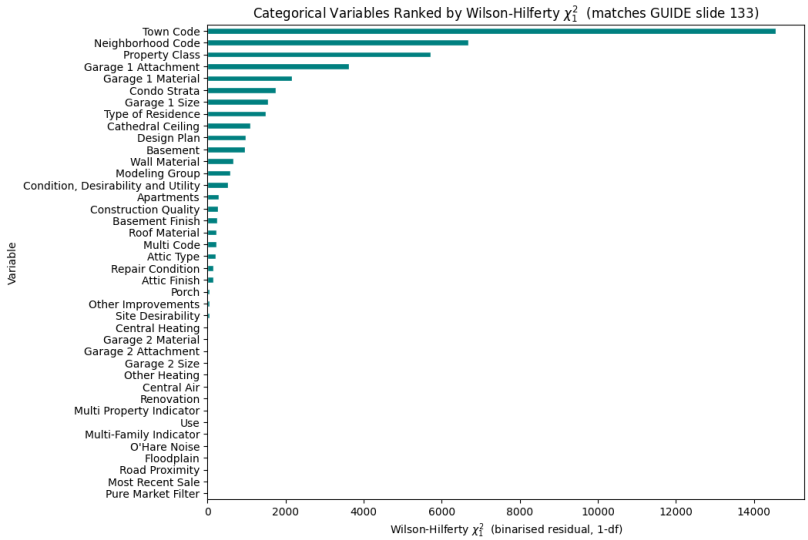




1.4.3 Categorical-predictor ranking: Wilson-Hilferty χ^2_1 (tertile target)



1.4.4 Categorical-predictor ranking: Wilson-Hilferty χ^2_1 (binarised-residual target)



1.4.5 Top-3 visual diagnostics: scatter plots vs log Sale Price

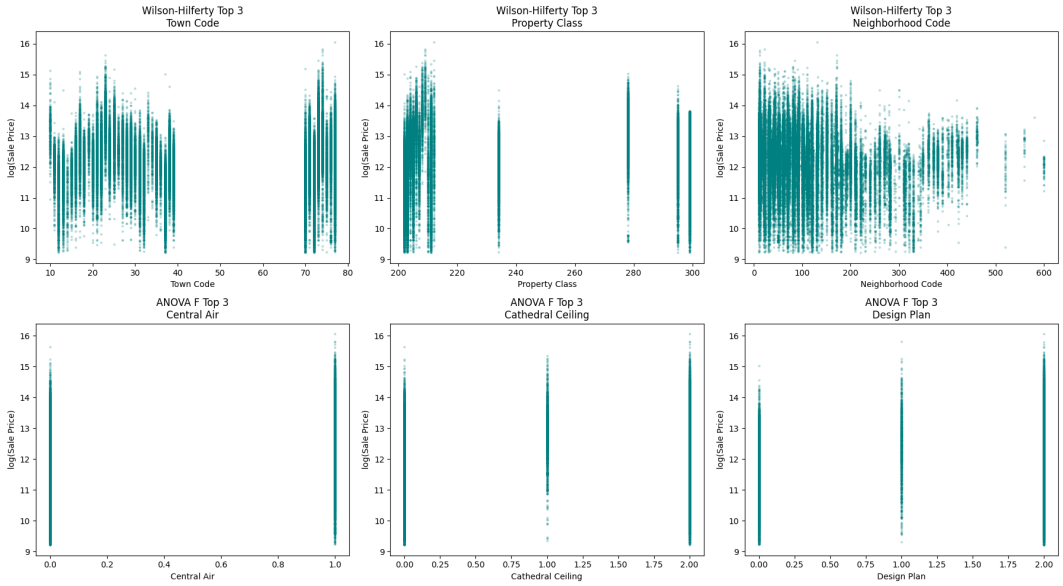


Figure 2: Top-3 categorical predictors irrespective of number of levels, ranked by Wilson-Hilferty (top-row) and ANOVA (bottom-row)

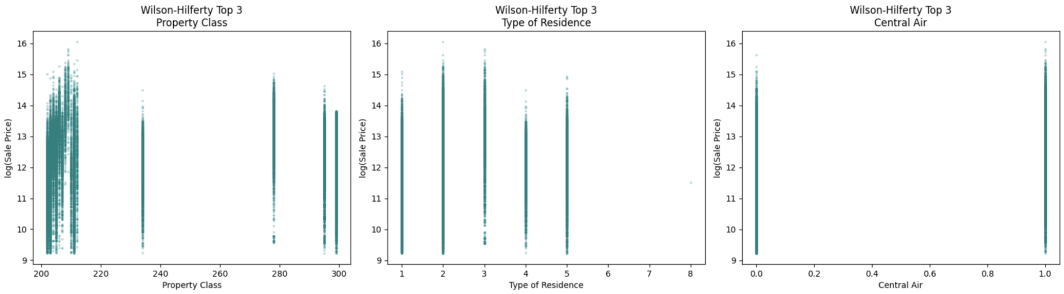


Figure 3: Top-3 categorical predictors with fewer than 20 levels, ranked by Wilson-Hilferty

1.4.6 Top-3 visual diagnostics: boxplots of Sale Price by category

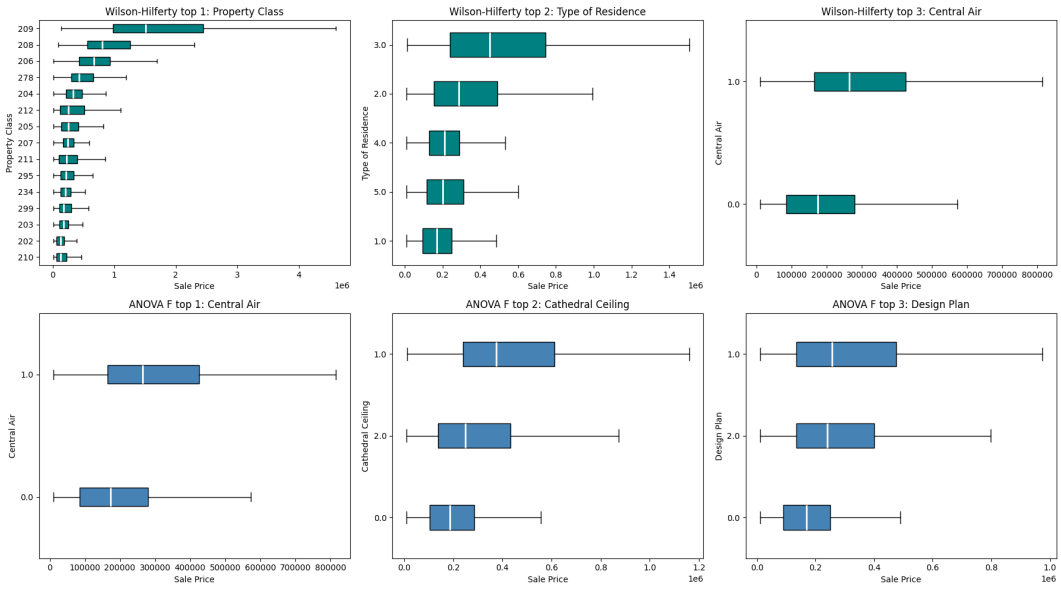


Figure 4: Top-3 categorical predictors with fewer than 20 levels, ranked by Wilson-Hilferty (top-row) and ANOVA (bottom-row)

The scatter plots (Figures 2 and 3) and boxplots (Figure 4) show that both Wilson-Hilferty and ANOVA rankings identify categorical predictors with clear separation in log(Sale Price). The top Wilson-Hilferty features (Town Code, Property Class, Neighborhood Code) exhibit particularly strong vertical banding and well-separated medians across multiple categories. In comparison, the top ANOVA features (Central Air, Cathedral Ceiling, Design Plan) also show clear separation between medians, but the high-ranked categories are much sparser, consisting primarily of binary or low-cardinality variables.

2 Methods

2.1 Evaluation Protocol

We use a strict *temporal* hold-out: training data are sales from 2013–2017, test data are sales from 2018–2019. This mirrors the way an assessor would deploy the model on next year's transactions and prevents look-ahead leakage that random splits would allow. Internally, GUIDE prunes via 10-fold cross-validation on the training window only.

Performance is summarised on the log scale by

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \quad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \quad R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

2.2 Linear Discriminant Analysis

We binned Sale Price into tertiles to create the target variable for LDA. We first dropped numerical columns with 10% or more missingness to prevent the subsequent row-wise deletion from drastically

reducing the size of the dataset, since structural missingness in a high-missingness column would otherwise force the removal of most rows. We then dropped any remaining rows that still contained missing values (rather than imputing), because median imputation could potentially violate LDA's within-class normality assumption. Categorical features were then one-hot encoded with NaN as its own level before fitting `LinearDiscriminantAnalysis`.

Cross-validation accuracy on the training set rose from **0.543** (numerical features only) to **0.588** after adding the one-hot-encoded categoricals. A two-proportion z-test on the difference in classification accuracy gave $z = 8.83$ and $p = 1.09 \times 10^{-18}$, confirming the improvement is statistically significant rather than by chance.

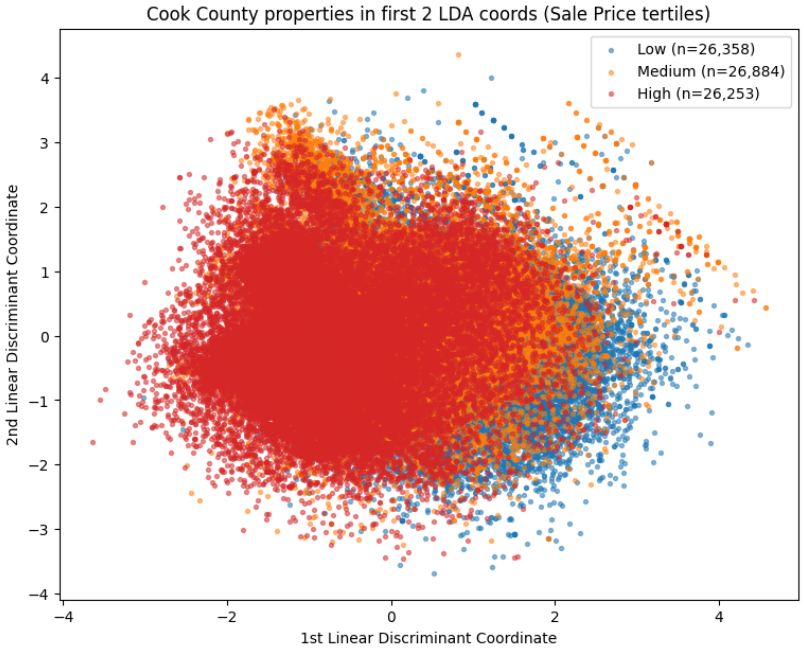


Figure 5: LDA

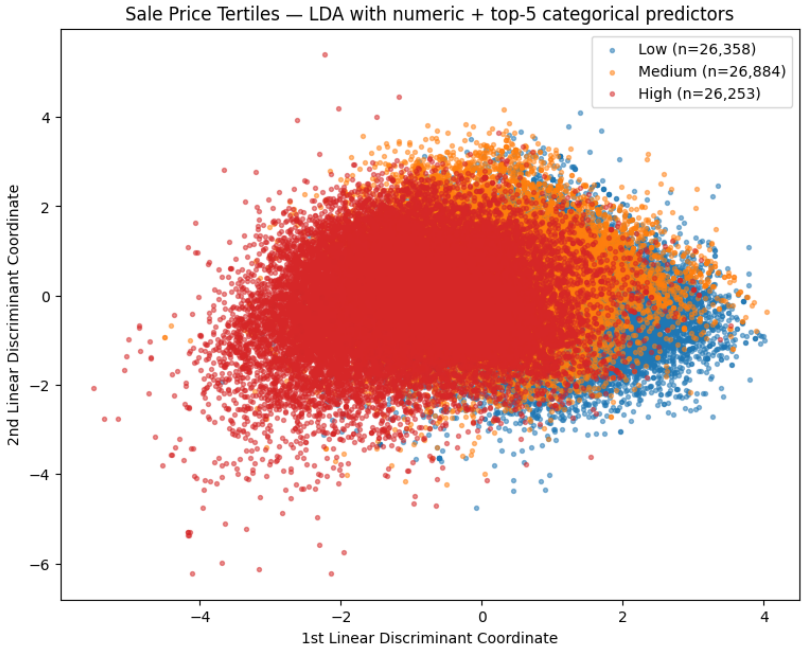


Figure 6: LDA

2.3 Linear and Ridge Regression

We constructed a scikit-learn pipeline consisting of median imputation for numerical features, standard scaling for the numerics, and one-hot encoding for categorical predictors (selected via the Wilson-Hilferty ranking). Ridge regression was fitted with regularization parameter $\alpha = 0.1$.

Table 2: Linear and Ridge Regression on 2018–2019 temporal hold-out

Model	**Log MSE**	**Log RMSE**	**Log R^2**	**n**
Linear regression	0.3537	0.5947	0.4633	23,810
Ridge regression ($\alpha = 0.1$)	0.3534	0.5945	0.4637	23,810

Table 3: Ridge Regression ($\alpha = 0.1$) using only GUIDE-ranked features – 2018–2019 temporal hold-out

Model	**Log MSE**	**Log RMSE**	**Log R^2**
Ridge (GUIDE-ranked features only)	0.3541	0.5950	0.4627

As a further experiment, we removed redundant features identified by combining high Pearson correlations with similar missingness patterns and retaining only the feature with higher GUIDE importance (a procedure we called `var_to_drop`). The dropped variables were `Bedrooms`, `Rooms`, `Full Baths`, `Apartments`, `Garage 1 Area`, `Garage 2 Size`, and `Garage 2 Attachment`.

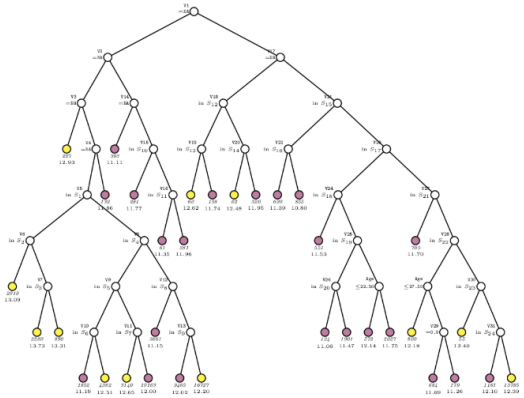
Table 4: Ridge Regression (GUIDE-importance ranked variables only, post dropping `var_to_drop` variables per Section 1.4.1) – 2018–2019 temporal hold-out

Model	**Log MSE**	**Log RMSE**	**Log R^2**
Ridge Regression	0.3546	0.5955	0.4618

2.4 GUIDE Regression Trees

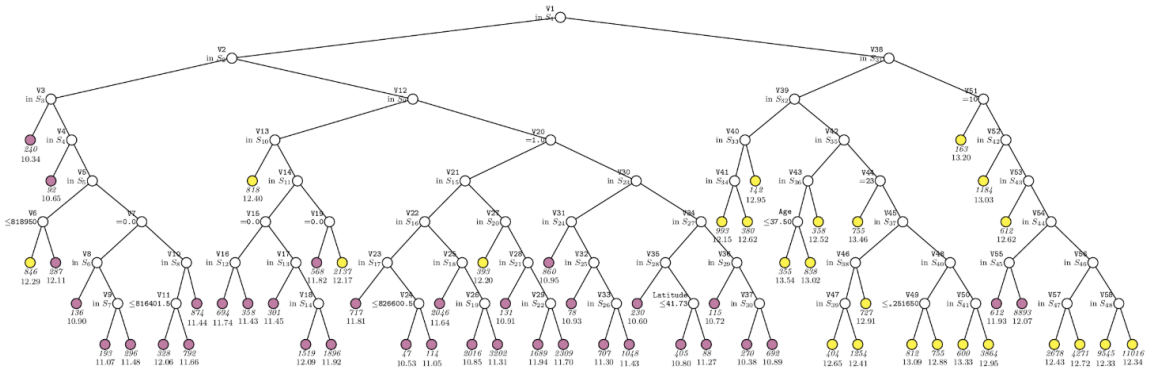
2.4.1 GUIDE Tree Progression

The five trees below trace the four comparisons that produced our primary GUIDE configuration. The GUIDE Random Forest tree is added in a later subsection.



GUIDE v.46.2 0.250-SE piecewise-stepwise linear least-squares regression tree (missing regressor values imputed and missing indicators added) for predicting `log.Sale.Price`. At each split, an observation goes to the left branch if and only if the condition is satisfied. V1 = `Number.of.Units`. V2 = `Total.Building.Square.Feet`. V3 = `Census.Tract`. V4 = `Percent.Ownership`. V5 = `Town.Code`. V6 = `Neighborhood.Code`. V7 = `Town.Code`. V8 = `Town.Code`. V9 = `Neighborhood.Code`. V10 = `Town.Code`. V11 = `Neighborhood.Code`. V12 = `Town.Code`. V13 = `Town.Code`. V14 = `Census.Tract`. V15 = `Neighborhood.Code`. V16 = `Neighborhood.Code`. V17 = `Census.Tract`. V18 = `Neighborhood.Code`. V19 = `Neighborhood.Code`. V20 = `Neighborhood.Code`. V21 = `Town.Code`. V22 = `Neighborhood.Code`. V23 = `Town.Code`. V24 = `Neighborhood.Code`. V25 = `Neighborhood.Code`. V26 = `Neighborhood.Code`. V27 = `Neighborhood.Code`. V28 = `Neighborhood.Code`. V29 = `Floodplain`. V30 = `Neighborhood.Code`. V31 = `Town.Code`. S_1 = {10, 23, 25, 27, 33, 73, 74}. S_2 = {10, 11, 14, 20, 30, 40, 50, 51, 52, 60, 61, 72, 90, 100, 102, 120, 131, 140, 150, 230, 240, 250, 260, 270, 280, 350, 400, 420, 430}. S_3 = {23, 73, 74}. S_4 = {14, 24, 26, 28, 29, 31, 71, 72, 75}. S_5 = {10, 11, 13, 21, 23, 33, 36, 40, 42, 43, 47, 60, 84, 91, 92, 100, 102, 110, 121, 140, 141, 152, 170, 171, 190, 194, 210, 212, 222, 223, 230, 251, 271, 312, 323, 330, 390, 410, 430, 440, 461}. S_6 = {14, 72}. S_7 = {12, 14, 15, 22, 31, 37, 39, 45, 46, 74, 81, 82, 83, 88, 90, 99, 101, 103, 161, 173, 174, 362, 402, 420, 432, 463, 560, 580, 600}. S_8 = {12, 37}. S_9 = {13, 17, 18, 20, 30, 32, 39, 76}. S_{10} = {11, 20, 22, 23, 31, 32, 40, 41, 43, 70, 74, 80, 90, 91, 92, 93, 100, 130, 141, 150, 171, 200, 210, 240, 251, 270, 350, 362, 430, 440, 461, 520}. S_{11} = {52, 71, 115, 132, 152, 330, 380}. S_{12} = {11, 12, 22, 30, 31, 33, 42, 50, 84, 91, 110}. S_{13} = {12, 42, 84, 110}. S_{14} = {43, 63, 80, 131, 152, 160}. S_{15} = {11, 12, 13, 14, 15, 32, 36, 37, 39}. S_{16} = {10, 41, 42, 50, 60, 71, 82, 90, 92, 100, 110, 161, 162, 170, 180, 190, 201, 210, 220, 230, 240, 270, 290, 310, 312, 330, 340, 350, 400, 410}. S_{17} = {16, 19, 20, 28, 29, 30, 34, 35, 70, 75}. S_{18} = {21, 51, 53, 70, 82, 96, 113, 121}. S_{19} = {10, 11, 22, 40, 50, 62, 74, 80, 81, 83, 85, 90, 100, 109, 111, 120, 140, 150, 151}. S_{20} = {74, 83, 111, 120, 140, 151}. S_{21} = {19, 53, 74, 85, 104, 112, 115, 121, 130, 140, 161, 166, 220, 270, 361, 362, 390, 402, 410, 430, 440, 520, 600}. S_{22} = {13, 14, 15, 23, 27, 65, 80, 90, 91, 103, 162, 180, 260, 371}. S_{23} = {24, 141}. S_{24} = {17, 22, 25, 33}. Sample size (in *italics*) and mean of `log.Sale.Price` printed below nodes. Terminal nodes with means above and below value of 12.15 at root node are painted yellow and purple respectively. Second best split variable at root node is `Neighborhood.Code`.

Figure 7: GUIDE tree under 10-fold cross-validation on the pooled 2013–2019 data, stepwise-linear leaves (`cook_county.in`).



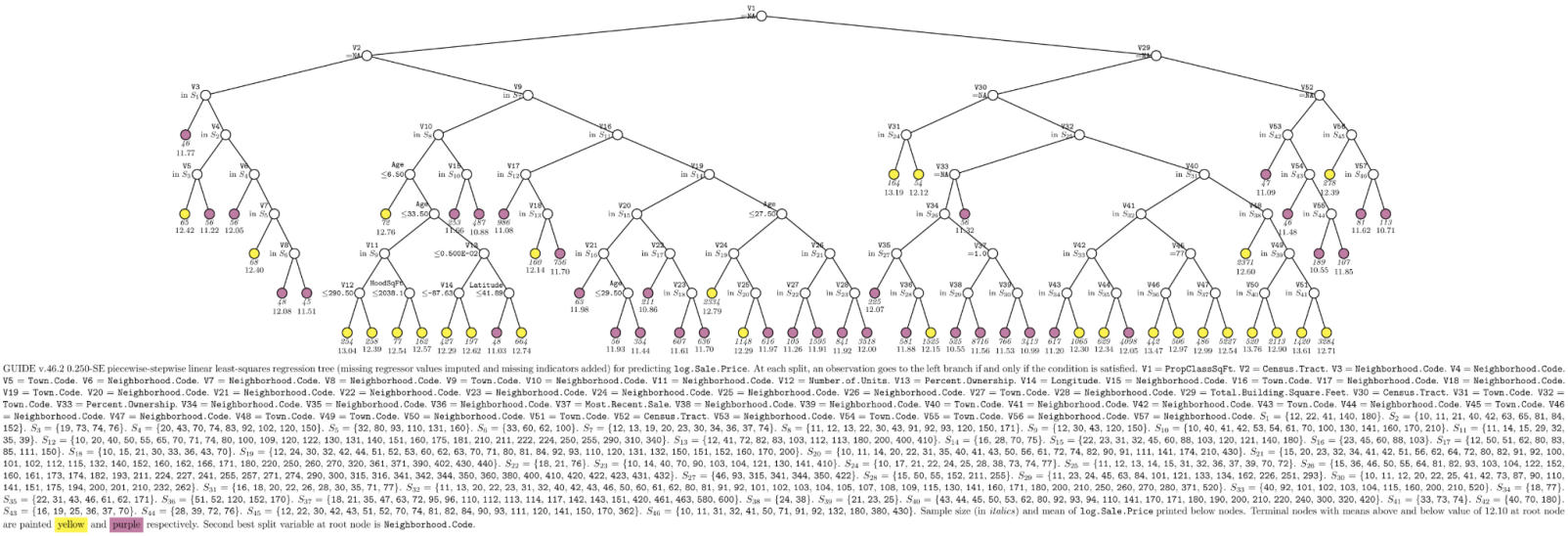
GUIDE v.46.2 0.250-SE least-squares multiple linear regression tree (missing regressor values imputed and missing indicators added) for predicting `log.Sale.Price`. At each split, an observation goes to the left branch if and only if the condition is satisfied. Symbol '$\leq_s$' stands for '$\leq$ or missing.' V1 = `Town.Code`. V2 = `Town.Code`. V3 = `Neighborhood.Code`. V4 = `Neighborhood.Code`. V5 = `Neighborhood.Code`. V6 = `Census.Tract`. V7 = `Most.Recent.Sale`. V8 = `Neighborhood.Code`. V9 = `Neighborhood.Code`. V10 = `Neighborhood.Code`. V11 = `Census.Tract`. V12 = `Neighborhood.Code`. V13 = `Neighborhood.Code`. V14 = `Neighborhood.Code`. V15 = `Most.Recent.Sale`. V16 = `Town.Code`. V17 = `Town.Code`. V18 = `Town.Code`. V19 = `Most.Recent.Sale`. V20 = `Most.Recent.Sale`. V21 = `Town.Code`. V22 = `Neighborhood.Code`. V23 = `Neighborhood.Code`. V24 = `Census.Tract`. V25 = `Neighborhood.Code`. V26 = `Neighborhood.Code`. V27 = `Neighborhood.Code`. V28 = `Neighborhood.Code`. V29 = `Neighborhood.Code`. V30 = `Neighborhood.Code`. V31 = `Town.Code`. V32 = `Neighborhood.Code`. V33 = `Neighborhood.Code`. V34 = `Neighborhood.Code`. V35 = `Neighborhood.Code`. V36 = `Neighborhood.Code`. V37 = `Neighborhood.Code`. V38 = `Neighborhood.Code`. V39 = `Town.Code`. V40 = `Neighborhood.Code`. V41 = `Neighborhood.Code`. V42 = `Town.Code`. V43 = `Neighborhood.Code`. V44 = `Town.Code`. V45 = `Town.Code`. V46 = `Neighborhood.Code`. V47 = `Town.Code`. V48 = `Neighborhood.Code`. V49 = `Census.Tract`. V50 = `Neighborhood.Code`. V51 = `Town.Code`. V52 = `Town.Code`. V53 = `Town.Code`. V54 = `Town.Code`. V55 = `Neighborhood.Code`. V56 = `Town.Code`. V57 = `Neighborhood.Code`. V58 = `Neighborhood.Code`. $S_1 = \{12, 13, 14, 15, 18, 20, 31, 32, 36, 37, 39, 70, 72, 75\}$. $S_2 = \{12, 14, 31\}$. $S_3 = \{101, 111, 112, 121, 122, 162\}$. $S_4 = \{84, 85, 88, 132\}$. $S_5 = \{46, 50, 51, 52, 53, 54, 71, 102, 103, 104, 110, 120, 141, 150, 151, 152, 160, 161, 170, 173, 174, 210\}$. $S_6 = \{31, 32, 40, 60, 61, 81, 86, 93, 191\}$. $S_7 = \{20, 30, 33, 43, 44, 63, 65, 72, 142, 190\}$. $S_8 = \{10, 21, 22, 31, 32, 40, 41, 42, 44, 72, 73, 80, 91, 92, 93, 191\}$. $S_9 = \{10, 11, 12, 13, 14, 15, 18, 19, 20, 25, 31, 35, 36, 41, 43, 50, 81, 82, 85, 90, 122, 151, 152, 182, 191, 193, 194, 200, 227, 232, 241, 255, 257, 262, 270, 271, 274, 300, 315, 316, 341, 344, 350, 360, 380, 400, 410, 420, 422, 423, 431, 432\}$. $S_{10} = \{14, 18, 19, 35, 36, 82, 182, 227, 257, 271, 300, 315, 316, 360, 400, 410, 420, 422, 431, 432\}$. $S_{11} = \{10, 11, 20, 25, 31, 43, 50, 85, 151, 152, 191, 200, 232, 255, 270, 341\}$. $S_{12} = \{18, 36, 39, 70, 75\}$. $S_{13} = \{13, 15, 37\}$. $S_{14} = \{39, 70, 75\}$. $S_{15} = \{32, 37, 70, 72\}$. $S_{16} = \{22, 30, 45, 54, 102, 140, 250, 260, 293\}$. $S_{17} = \{30, 250\}$. $S_{18} = \{40, 42, 53, 63, 103, 110, 150, 160, 161, 162, 175, 185, 192, 201, 212, 220, 222, 224, 230, 240, 280, 282, 310, 320, 323, 361\}$. $S_{19} = \{21, 23, 24, 51, 83, 91, 101, 120, 121, 134, 163, 171, 180, 210, 251, 312, 330, 345\}$. $S_{20} = \{21, 23, 33, 75, 101, 170, 180, 280\}$. $S_{21} = \{91, 120, 131, 133, 220, 320, 340\}$. $S_{22} = \{30, 70, 80, 100, 110, 130, 312, 330\}$. $S_{23} = \{22, 30, 32, 40, 42, 55, 60, 65, 70, 71, 72, 80, 83, 92, 100, 103, 110, 130, 150, 160, 161, 162, 170, 175, 183, 185, 190, 192, 201, 210, 211, 224, 230, 240, 250, 280, 290, 293, 310, 314, 323, 361\}$. $S_{24} = \{37, 70\}$. $S_{25} = \{92, 190, 240, 293\}$. $S_{26} = \{22, 42, 65, 80, 100, 130, 160, 161, 170, 175, 224, 250, 310, 314, 323\}$. $S_{27} = \{21, 52, 61, 120, 133, 134, 181, 220, 222, 226, 251, 282, 312\}$. $S_{28} = \{120, 133, 134, 226, 251\}$. $S_{29} = \{23, 45, 102, 132, 223, 281\}$. $S_{30} = \{54, 63, 75, 91, 101, 121, 321, 330\}$. $S_{31} = \{12, 21, 24, 25, 34, 35, 38, 44, 45, 48, 51, 64, 67, 71, 72, 75, 83, 84, 87, 92, 93, 94, 95, 106, 110, 114, 116, 117, 120, 131, 132, 133, 142, 143, 150, 151, 152, 160, 165, 170, 190, 230, 240, 300, 310, 361, 400, 420, 440, 461, 463, 560, 580\}$. $S_{32} = \{16, 30, 35, 38\}$. $S_{33} = \{12, 51, 71, 72, 94, 110, 116, 160, 170\}$. $S_{34} = \{12, 51, 110, 170\}$. $S_{35} = \{21, 29, 74\}$. $S_{36} = \{12, 64, 67, 71, 110, 120\}$. $S_{37} = \{17, 19, 24, 28, 71\}$. $S_{38} = \{45, 51, 71, 120, 150, 440\}$. $S_{39} = \{17, 24\}$. $S_{40} = \{25, 93, 132, 142, 150, 240, 310\}$. $S_{41} = \{21, 24, 34, 170, 190, 230, 300, 400, 420\}$. $S_{42} = \{23, 26, 33\}$. $S_{43} = \{19, 34\}$. $S_{44} = \{11, 16, 21, 30, 35, 77\}$. $S_{45} = \{20, 63\}$. $S_{46} = \{17, 24, 25, 27, 74\}$. $S_{47} = \{22, 31, 32, 40, 81, 112, 130, 220, 320, 430\}$. $S_{48} = \{10, 13, 14, 22, 27, 30, 31, 32, 36, 52, 56, 60, 61, 63, 74, 81, 82, 85, 130, 140, 171, 210, 270, 362, 371, 390, 520, 600\}$. Sample size (in *italics*) and mean of `log.Sale.Price` printed below nodes. Terminal nodes with means above and below value of 12.15 at root node are painted yellow and purple respectively. Second best split variable at root node is `Neighborhood.Code`.

Figure 8: GUIDE tree under 10-fold cross-validation on the pooled 2013–2019 data, multiple-linear leaves (`cook_county.in`).

Table 5: CV stepwise vs multiple-linear leaf models (pre-temporal-split data, 10-fold CV).

Model	**#Tnodes**	**CV MSE**	**CV RMSE**	**CV R^2**	**Final (resub) R^2**
Stepwise	34	0.2594	0.5093	0.676	0.6192
Multiple-linear	61	0.3410	0.5839	0.574	0.7002

Comparison 1 – CV stepwise vs CV multiple-linear (pre-temporal-split data).



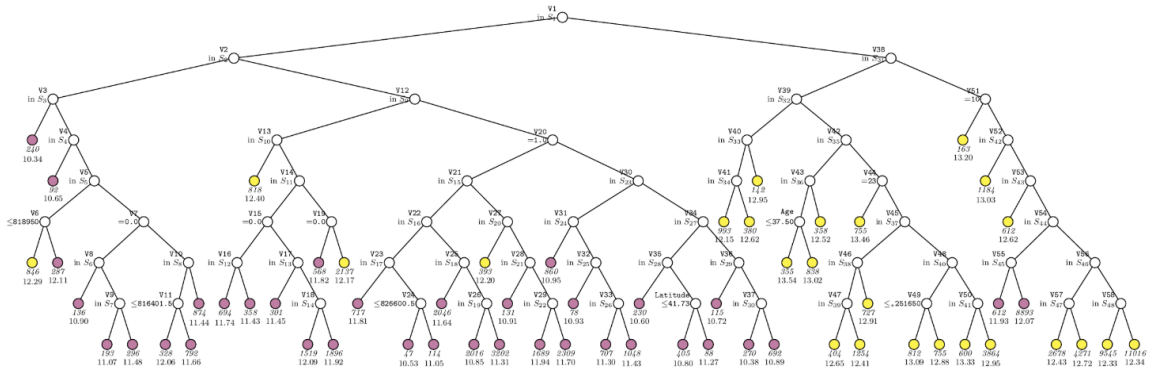
GUIDE v.46.2 0.250-SE piecewise-stepwise linear least-squares regression tree (missing regressor values imputed and missing indicators added) for predicting `log.Sale.Price`. At each split, an observation goes to the left branch if and only if the condition is satisfied. `V1` = `PropClassSqFt`. `V2` = `Census.Tract`. `V3` = `Neighborhood.Code`. `V4` = `Neighborhood.Code`. `V5` = `Town.Code`. `V6` = `Neighborhood.Code`. `V7` = `Neighborhood.Code`. `V8` = `Neighborhood.Code`. `V9` = `Town.Code`. `V10` = `Neighborhood.Code`. `V11` = `Neighborhood.Code`. `V12` = `Number.of.Units`. `V13` = `Percent.Ownership`. `V14` = `Longitude`. `V15` = `Neighborhood.Code`. `V16` = `Town.Code`. `V17` = `Neighborhood.Code`. `V18` = `Neighborhood.Code`. `V19` = `Town.Code`. `V20` = `Neighborhood.Code`. `V21` = `Neighborhood.Code`. `V22` = `Neighborhood.Code`. `V23` = `Neighborhood.Code`. `V24` = `Neighborhood.Code`. `V25` = `Neighborhood.Code`. `V26` = `Neighborhood.Code`. `V27` = `Town.Code`. `V28` = `Neighborhood.Code`. `V29` = `Total.Building.Square.Feet`. `V30` = `Census.Tract`. `V31` = `Town.Code`. `V32` = `Town.Code`. `V33` = `Percent.Ownership`. `V34` = `Neighborhood.Code`. `V35` = `Neighborhood.Code`. `V36` = `Neighborhood.Code`. `V37` = `Most.Recent.Sale`. `V38` = `Neighborhood.Code`. `V39` = `Neighborhood.Code`. `V40` = `Town.Code`. `V41` = `Neighborhood.Code`. `V42` = `Neighborhood.Code`. `V43` = `Town.Code`. `V44` = `Neighborhood.Code`. `V45` = `Town.Code`. `V46` = `Neighborhood.Code`. `V47` = `Neighborhood.Code`. `V48` = `Town.Code`. `V49` = `Town.Code`. `V50` = `Neighborhood.Code`. `V51` = `Town.Code`. `V52` = `Census.Tract`. `V53` = `Neighborhood.Code`. `V54` = `Town.Code`. `V55` = `Town.Code`. `V56` = `Neighborhood.Code`. `V57` = `Neighborhood.Code`. S_1 = {12, 22, 41, 140, 180}. S_2 = {10, 11, 21, 40, 42, 63, 65, 81, 84, 152}. S_3 = {19, 73, 74, 76}. S_4 = {20, 43, 70, 74, 83, 92, 102, 120, 150}. S_5 = {32, 80, 93, 110, 131, 160}. S_6 = {33, 60, 62, 100}. S_7 = {12, 13, 19, 20, 23, 30, 34, 36, 37, 74}. S_8 = {11, 12, 13, 22, 30, 43, 91, 92, 93, 120, 150, 171}. S_9 = {12, 30, 43, 120, 150}. S_{10} = {10, 40, 41, 42, 53, 54, 61, 70, 100, 130, 141, 160, 170, 210}. S_{11} = {11, 14, 15, 29, 32, 35, 39}. S_{12} = {10, 20, 40, 50, 55, 65, 70, 71, 74, 80, 100, 109, 120, 122, 130, 131, 140, 151, 160, 175, 181, 210, 211, 222, 224, 250, 255, 290, 310, 340}. S_{13} = {12, 41, 72, 82, 83, 103, 112, 113, 180, 200, 400, 410}. S_{14} = {16, 28, 70, 75}. S_{15} = {22, 23, 31, 32, 45, 60, 88, 103, 120, 121, 140, 180}. S_{16} = {23, 45, 60, 88, 103}. S_{17} = {12, 50, 51, 62, 80, 83, 85, 111, 150}. S_{18} = {10, 15, 21, 30, 33, 36, 43, 70}. S_{19} = {12, 24, 30, 32, 42, 44, 51, 52, 53, 60, 62, 63, 70, 71, 80, 81, 84, 92, 93, 110, 120, 131, 132, 150, 151, 152, 160, 170, 200}. S_{20} = {10, 11, 14, 20, 22, 31, 35, 40, 41, 43, 50, 56, 61, 72, 74, 82, 90, 91, 111, 141, 174, 210, 430}. S_{21} = {15, 20, 23, 32, 34, 41, 42, 51, 56, 62, 64, 72, 80, 82, 91, 92, 100, 101, 102, 112, 115, 132, 140, 152, 160, 162, 166, 171, 180, 220, 250, 260, 270, 320, 361, 371, 390, 402, 430, 440}. S_{22} = {18, 21, 76}. S_{23} = {10, 14, 40, 70, 90, 103, 104, 121, 130, 141, 410}. S_{24} = {10, 17, 21, 22, 24, 25, 28, 38, 73, 74, 77}. S_{25} = {11, 12, 13, 14, 15, 31, 32, 36, 37, 39, 70, 72}. S_{26} = {15, 36, 46, 50, 55, 64, 81, 82, 93, 103, 104, 122, 152, 160, 161, 173, 174, 182, 193, 211, 224, 227, 241, 255, 257, 271, 274, 290, 300, 315, 316, 341, 342, 344, 350, 360, 380, 400, 410, 420, 422, 423, 431, 432}. S_{27} = {46, 93, 315, 341, 344, 350, 422}. S_{28} = {15, 50, 55, 152, 211, 255}. S_{29} = {11, 23, 24, 45, 63, 84, 101, 121, 133, 134, 162, 226, 251, 293}. S_{30} = {10, 11, 12, 20, 22, 25, 41, 42, 73, 87, 90, 110, 141, 151, 175, 194, 200, 201, 210, 232, 262}. S_{31} = {16, 18, 20, 22, 26, 28, 30, 35, 71, 77}. S_{32} = {11, 13, 20, 22, 23, 31, 32, 40, 42, 43, 46, 50, 60, 61, 62, 80, 81, 91, 92, 101, 102, 103, 104, 105, 107, 108, 109, 115, 130, 141, 160, 171, 180, 200, 210, 250, 260, 270, 280, 371, 520}. S_{33} = {40, 92, 101, 102, 103, 104, 115, 160, 200, 210, 520}. S_{34} = {18, 77}. S_{35} = {22, 31, 43, 46, 61, 62, 171}. S_{36} = {51, 52, 120, 152, 170}. S_{37} = {18, 21, 35, 47, 63, 72, 95, 96, 110, 112, 113, 114, 117, 142, 143, 151, 420, 461, 463, 580, 600}. S_{38} = {24, 38}. S_{39} = {21, 23, 25}. S_{40} = {43, 44, 45, 50, 53, 62, 80, 92, 93, 94, 110, 141, 170, 171, 180, 190, 200, 210, 220, 240, 300, 320, 420}. S_{41} = {33, 73, 74}. S_{42} = {40, 70, 180}. S_{43} = {16, 19, 25, 36, 37, 70}. S_{44} = {28, 39, 72, 76}. S_{45} = {12, 22, 30, 42, 43, 51, 52, 70, 74, 81, 82, 84, 90, 93, 111, 120, 141, 150, 170, 362}. S_{46} = {10, 11, 31, 32, 41, 50, 71, 91, 92, 132, 180, 380, 430}. Sample size (in *italics*) and mean of `log.Sale.Price` printed below nodes. Terminal nodes with means above and below value of 12.10 at root node are painted yellow and purple respectively. Second best split variable at root node is `Neighborhood.Code`.

Figure 9: GUIDE tree on the 2013–2017 / 2018–2019 temporal hold-out, stepwise-linear leaves, v3 feature set (`cook_county_train_v3.in`). Compare against Figure 7.

Table 6: Stepwise-linear leaves: 10-fold CV vs Temporal Hold-out (2018–2019)

Version	**#Tnodes**	**Log MSE**	**Log RMSE**	**Log R^2**	**n**
10-fold CV	34	0.2594	0.5093	0.676	–
Temporal Hold-out	–	0.2315	0.4811	0.6487	23,810

Comparison 2 – best CV vs temporal hold-out (same leaf model).



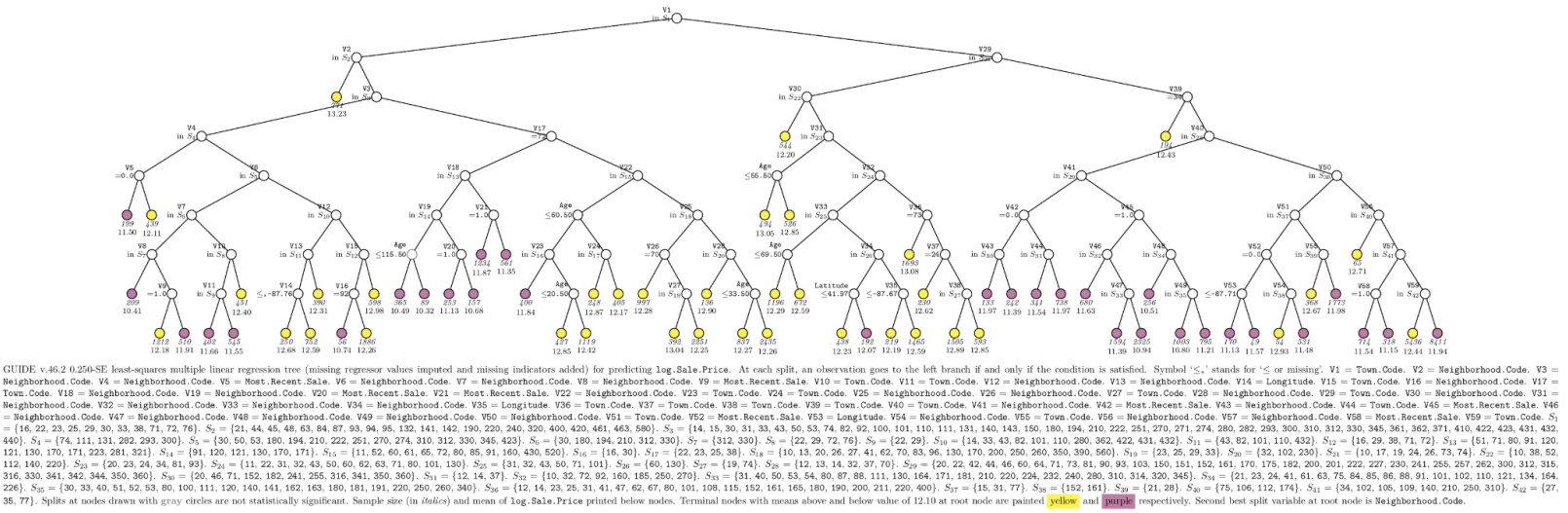
GUIDE v.46.2 0.250-SE least-squares multiple linear regression tree (missing regressor values imputed and missing indicators added) for predicting `log.Sale.Price`. At each split, an observation goes to the left branch if and only if the condition is satisfied. Symbol '$\leq_s$' stands for '$\leq$ or missing.' V1 = `Town.Code`. V2 = `Town.Code`. V3 = `Neighborhood.Code`. V4 = `Neighborhood.Code`. V5 = `Neighborhood.Code`. V6 = `Census.Tract`. V7 = `Most.Recent.Sale`. V8 = `Neighborhood.Code`. V9 = `Neighborhood.Code`. V10 = `Neighborhood.Code`. V11 = `Census.Tract`. V12 = `Neighborhood.Code`. V13 = `Neighborhood.Code`. V14 = `Neighborhood.Code`. V15 = `Most.Recent.Sale`. V16 = `Town.Code`. V17 = `Town.Code`. V18 = `Town.Code`. V19 = `Most.Recent.Sale`. V20 = `Most.Recent.Sale`. V21 = `Town.Code`. V22 = `Neighborhood.Code`. V23 = `Neighborhood.Code`. V24 = `Census.Tract`. V25 = `Neighborhood.Code`. V26 = `Neighborhood.Code`. V27 = `Neighborhood.Code`. V28 = `Neighborhood.Code`. V29 = `Neighborhood.Code`. V30 = `Neighborhood.Code`. V31 = `Town.Code`. V32 = `Neighborhood.Code`. V33 = `Neighborhood.Code`. V34 = `Neighborhood.Code`. V35 = `Neighborhood.Code`. V36 = `Neighborhood.Code`. V37 = `Neighborhood.Code`. V38 = `Neighborhood.Code`. V39 = `Town.Code`. V40 = `Neighborhood.Code`. V41 = `Neighborhood.Code`. V42 = `Town.Code`. V43 = `Neighborhood.Code`. V44 = `Town.Code`. V45 = `Town.Code`. V46 = `Neighborhood.Code`. V47 = `Town.Code`. V48 = `Neighborhood.Code`. V49 = `Census.Tract`. V50 = `Neighborhood.Code`. V51 = `Town.Code`. V52 = `Town.Code`. V53 = `Town.Code`. V54 = `Town.Code`. V55 = `Neighborhood.Code`. V56 = `Town.Code`. V57 = `Neighborhood.Code`. V58 = `Neighborhood.Code`. $S_1 = \{12, 13, 14, 15, 18, 20, 31, 32, 36, 37, 39, 70, 72, 75\}$. $S_2 = \{12, 14, 31\}$. $S_3 = \{101, 111, 112, 121, 122, 162\}$. $S_4 = \{84, 85, 88, 132\}$. $S_5 = \{46, 50, 51, 52, 53, 54, 71, 102, 103, 104, 110, 120, 141, 150, 151, 152, 160, 161, 170, 173, 174, 210\}$. $S_6 = \{31, 32, 40, 60, 61, 81, 86, 93, 191\}$. $S_7 = \{20, 30, 33, 43, 44, 63, 65, 72, 142, 190\}$. $S_8 = \{10, 21, 22, 31, 32, 40, 41, 42, 44, 72, 73, 80, 91, 92, 93, 191\}$. $S_9 = \{10, 11, 12, 13, 14, 15, 18, 19, 20, 25, 31, 35, 36, 41, 43, 50, 81, 82, 85, 90, 122, 151, 152, 182, 191, 193, 194, 200, 227, 232, 241, 255, 257, 262, 270, 271, 274, 300, 315, 316, 341, 344, 350, 360, 380, 400, 410, 420, 422, 423, 431, 432\}$. $S_{10} = \{14, 18, 19, 35, 36, 82, 182, 227, 257, 271, 300, 315, 316, 360, 400, 410, 420, 422, 431, 432\}$. $S_{11} = \{10, 11, 20, 25, 31, 43, 50, 85, 151, 152, 191, 200, 232, 255, 270, 341\}$. $S_{12} = \{18, 36, 39, 70, 75\}$. $S_{13} = \{13, 15, 37\}$. $S_{14} = \{39, 70, 75\}$. $S_{15} = \{32, 37, 70, 72\}$. $S_{16} = \{22, 30, 45, 54, 102, 140, 250, 260, 293\}$. $S_{17} = \{30, 250\}$. $S_{18} = \{40, 42, 53, 63, 103, 110, 150, 160, 161, 162, 175, 185, 192, 201, 212, 220, 222, 224, 230, 240, 280, 282, 310, 320, 323, 361\}$. $S_{19} = \{21, 23, 24, 51, 83, 91, 101, 120, 121, 134, 163, 171, 180, 210, 251, 312, 330, 345\}$. $S_{20} = \{21, 23, 33, 75, 101, 170, 180, 280\}$. $S_{21} = \{91, 120, 131, 133, 220, 320, 340\}$. $S_{22} = \{30, 70, 80, 100, 110, 130, 312, 330\}$. $S_{23} = \{22, 30, 32, 40, 42, 55, 60, 65, 70, 71, 72, 80, 83, 92, 100, 103, 110, 130, 150, 160, 161, 162, 170, 175, 183, 185, 190, 192, 201, 210, 211, 224, 230, 240, 250, 280, 290, 293, 310, 314, 323, 361\}$. $S_{24} = \{37, 70\}$. $S_{25} = \{92, 190, 240, 293\}$. $S_{26} = \{22, 42, 65, 80, 100, 130, 160, 161, 170, 175, 224, 250, 310, 314, 323\}$. $S_{27} = \{21, 52, 61, 120, 133, 134, 181, 220, 222, 226, 251, 282, 312\}$. $S_{28} = \{120, 133, 134, 226, 251\}$. $S_{29} = \{23, 45, 102, 132, 223, 281\}$. $S_{30} = \{54, 63, 75, 91, 101, 121, 321, 330\}$. $S_{31} = \{12, 21, 24, 25, 34, 35, 38, 44, 45, 48, 51, 64, 67, 71, 72, 75, 83, 84, 87, 92, 93, 94, 95, 106, 110, 114, 116, 117, 120, 131, 132, 133, 142, 143, 150, 151, 152, 160, 165, 170, 190, 230, 240, 300, 310, 361, 400, 420, 440, 461, 463, 560, 580\}$. $S_{32} = \{16, 30, 35, 38\}$. $S_{33} = \{12, 51, 71, 72, 94, 110, 116, 160, 170\}$. $S_{34} = \{12, 51, 110, 170\}$. $S_{35} = \{21, 29, 74\}$. $S_{36} = \{12, 64, 67, 71, 110, 120\}$. $S_{37} = \{17, 19, 24, 28, 71\}$. $S_{38} = \{45, 51, 71, 120, 150, 440\}$. $S_{39} = \{17, 24\}$. $S_{40} = \{25, 93, 132, 142, 150, 240, 310\}$. $S_{41} = \{21, 24, 34, 170, 190, 230, 300, 400, 420\}$. $S_{42} = \{23, 26, 33\}$. $S_{43} = \{19, 34\}$. $S_{44} = \{11, 16, 21, 30, 35, 77\}$. $S_{45} = \{20, 63\}$. $S_{46} = \{17, 24, 25, 27, 74\}$. $S_{47} = \{22, 31, 32, 40, 81, 112, 130, 220, 320, 430\}$. $S_{48} = \{10, 13, 14, 22, 27, 30, 31, 32, 36, 52, 56, 60, 61, 63, 74, 81, 82, 85, 130, 140, 171, 210, 270, 362, 371, 390, 520, 600\}$. Sample size (in *italics*) and mean of `log.Sale.Price` printed below nodes. Terminal nodes with means above and below value of 12.15 at root node are painted yellow and purple respectively. Second best split variable at root node is `Neighborhood.Code`.

Figure 10: GUIDE tree on the temporal hold-out with the v2 feature set: `HoodPriceSqFt` and `PropClassPriceSqFt` replace raw sqft features (`cook_county_train_v2.in`). Compare against the v3 tree in Figure 9.

Table 7: GUIDE stepwise-linear leaves: v2 (price-per-sqft) vs v3 (raw square footage) on 2018–2019 temporal hold-out

Feature Set	**#Tnodes**	**Log MSE**	**Log RMSE**	**Log R^2**	**n**
v2 (HoodPriceSqFt + PropClassPriceSqFt)	–	0.2462	0.4962	0.6264	23,810
v3 (HoodSqFt + PropClassSqFt)	–	0.2315	0.4811	0.6487	23,810

Comparison 3 – feature engineering: price-per-sqft (v2) vs raw sqft (v3).



GUIDE v.46.2 0.250-SE least-squares multiple linear regression tree (missing regressor values imputed and missing indicators added) for predicting `log.Sale.Price`. At each split, an observation goes to the left branch if and only if the condition is satisfied. Symbol '$\leq_s$' stands for '$\leq$ or missing'. V1 = `Town.Code`. V2 = `Neighborhood.Code`. V3 = `Neighborhood.Code`. V4 = `Neighborhood.Code`. V5 = `Most.Recent.Sale`. V6 = `Neighborhood.Code`. V7 = `Neighborhood.Code`. V8 = `Neighborhood.Code`. V9 = `Most.Recent.Sale`. V10 = `Town.Code`. V11 = `Town.Code`. V12 = `Neighborhood.Code`. V13 = `Neighborhood.Code`. V14 = `Longitude`. V15 = `Town.Code`. V16 = `Neighborhood.Code`. V17 = `Town.Code`. V18 = `Neighborhood.Code`. V19 = `Neighborhood.Code`. V20 = `Most.Recent.Sale`. V21 = `Most.Recent.Sale`. V22 = `Neighborhood.Code`. V23 = `Town.Code`. V24 = `Town.Code`. V25 = `Neighborhood.Code`. V26 = `Neighborhood.Code`. V27 = `Town.Code`. V28 = `Neighborhood.Code`. V29 = `Town.Code`. V30 = `Neighborhood.Code`. V31 = `Neighborhood.Code`. V32 = `Neighborhood.Code`. V33 = `Neighborhood.Code`. V34 = `Neighborhood.Code`. V35 = `Longitude`. V36 = `Town.Code`. V37 = `Town.Code`. V38 = `Town.Code`. V39 = `Town.Code`. V40 = `Town.Code`. V41 = `Neighborhood.Code`. V42 = `Most.Recent.Sale`. V43 = `Neighborhood.Code`. V44 = `Town.Code`. V45 = `Most.Recent.Sale`. V46 = `Neighborhood.Code`. V47 = `Neighborhood.Code`. V48 = `Neighborhood.Code`. V49 = `Neighborhood.Code`. V50 = `Neighborhood.Code`. V51 = `Town.Code`. V52 = `Most.Recent.Sale`. V53 = `Longitude`. V54 = `Neighborhood.Code`. V55 = `Town.Code`. V56 = `Neighborhood.Code`. V57 = `Neighborhood.Code`. V58 = `Most.Recent.Sale`. V59 = `Town.Code`. S_1 = {16, 22, 23, 25, 29, 30, 33, 38, 71, 72, 76}. S_2 = {21, 44, 45, 48, 63, 84, 87, 93, 94, 95, 132, 141, 142, 190, 220, 240, 320, 400, 420, 461, 463, 580}. S_3 = {14, 15, 30, 31, 33, 43, 50, 53, 74, 82, 92, 100, 101, 110, 111, 131, 140, 143, 150, 180, 194, 210, 222, 251, 270, 271, 274, 280, 282, 293, 300, 310, 312, 330, 345, 361, 362, 371, 410, 422, 423, 431, 432, 440}. S_4 = {74, 111, 131, 282, 293, 300}. S_5 = {30, 50, 53, 180, 194, 210, 222, 251, 270, 274, 310, 312, 330, 345, 423}. S_6 = {30, 180, 194, 210, 312, 330}. S_7 = {312, 330}. S_8 = {22, 29, 72, 76}. S_9 = {22, 29}. S_{10} = {14, 33, 43, 82, 101, 110, 280, 362, 422, 431, 432}. S_{11} = {43, 82, 101, 110, 432}. S_{12} = {16, 29, 38, 71, 72}. S_{13} = {51, 71, 80, 91, 120, 121, 130, 170, 171, 223, 281, 321}. S_{14} = {91, 120, 121, 130, 170, 171}. S_{15} = {11, 52, 60, 61, 65, 72, 80, 85, 91, 160, 430, 520}. S_{16} = {16, 30}. S_{17} = {22, 23, 25, 38}. S_{18} = {10, 13, 20, 26, 27, 41, 62, 70, 83, 96, 130, 170, 200, 250, 260, 350, 390, 560}. S_{19} = {23, 25, 29, 33}. S_{20} = {32, 102, 230}. S_{21} = {10, 17, 19, 24, 26, 73, 74}. S_{22} = {10, 38, 52, 112, 140, 220}. S_{23} = {20, 23, 24, 34, 81, 93}. S_{24} = {11, 22, 31, 32, 43, 50, 60, 62, 63, 71, 80, 101, 130}. S_{25} = {31, 32, 43, 50, 71, 101}. S_{26} = {60, 130}. S_{27} = {19, 74}. S_{28} = {12, 13, 14, 32, 37, 70}. S_{29} = {20, 22, 42, 44, 46, 60, 64, 71, 73, 81, 90, 93, 103, 150, 151, 152, 161, 170, 175, 182, 200, 201, 222, 227, 230, 241, 255, 257, 262, 300, 312, 315, 316, 330, 341, 342, 344, 350, 360}. S_{30} = {20, 46, 71, 152, 182, 241, 255, 316, 341, 350, 360}. S_{31} = {12, 14, 37}. S_{32} = {10, 32, 72, 92, 160, 185, 250, 270}. S_{33} = {31, 40, 50, 53, 54, 80, 87, 88, 111, 130, 164, 171, 181, 210, 220, 224, 232, 240, 280, 310, 314, 320, 345}. S_{34} = {21, 23, 24, 41, 61, 63, 75, 84, 85, 86, 88, 91, 101, 102, 110, 121, 134, 164, 226}. S_{35} = {30, 33, 40, 51, 52, 53, 80, 100, 111, 120, 140, 141, 162, 163, 180, 181, 191, 220, 250, 260, 340}. S_{36} = {12, 14, 23, 25, 31, 41, 47, 62, 67, 80, 101, 108, 115, 152, 161, 165, 180, 190, 200, 211, 220, 400}. S_{37} = {15, 31, 77}. S_{38} = {152, 161}. S_{39} = {21, 28}. S_{40} = {75, 106, 112, 174}. S_{41} = {34, 102, 105, 109, 140, 210, 250, 310}. S_{42} = {27, 35, 77}. Splits at nodes drawn with gray circles are not statistically significant. Sample size (in *italics*) and mean of `log.Sale.Price` printed below nodes. Terminal nodes with means above and below value of 12.10 at root node are painted yellow and purple respectively. Second best split variable at root node is `Neighborhood.Code`.

Figure 11: GUIDE tree on the v3 temporal hold-out with multiple-linear leaves (`cook_county_train_linear.in`). Compare against the stepwise-linear v3 tree in Figure 9.

Table 8: GUIDE v3 feature set: Multiple-linear vs Stepwise-linear leaves on 2018–2019 temporal hold-out

Leaf Model	**#Tnodes**	**Log MSE**	**Log RMSE**	**Log R^2**	**n**
Multiple-linear	–	0.2328	0.4825	0.6468	23,810
Stepwise-linear	–	0.2315	0.4811	0.6487	23,810

Comparison 4 – stepwise-linear vs multiple-linear at the best version (v3).

GUIDE Random Forest. *Note:* GUIDE's random forest mode does not generate an R script that can be executed on new data. Because of this, we were unable to apply the trained forest to the 2018-2019 test set. The numbers below are therefore reported on the training window (2013–2017).

Table 9: GUIDE Random Forest: Resubstitution vs OOB estimate (temporal training window 2013–2017)

Estimate	**#Tnodes**	**Log MSE**	**Log RMSE**	**Log R^2**	**n**
Resubstitution (training)	–	0.1753	0.4187	0.7937	56,933
OOB (honest)	–	0.1970	0.4438	0.7682	56,933

2.5 rpart (CART in R)

We fitted the `rpart` implementation of CART in R using `method="anova"`, `cp = 0.001`, `xval = 5`, and `minsplit = 20`. Columns with more than 32 levels were dropped to satisfy rpart's regression-mode limit.

We have used similar parameters across all models to provide a comparable benchmark. Setting minsplit to 20 and cp = 0.001 helps us ensure that we do not inflate the tree with many splits and at the same time, we do not increase bias too much by reducing splits too aggressively.

Table 10: rpart (CART in R) – 66-leaf tree on 2018–2019 temporal hold-out

Model	Log MSE	Log RMSE	Log R^2	n
rpart (CART in R)	0.3265	0.5714	0.5045	23,810

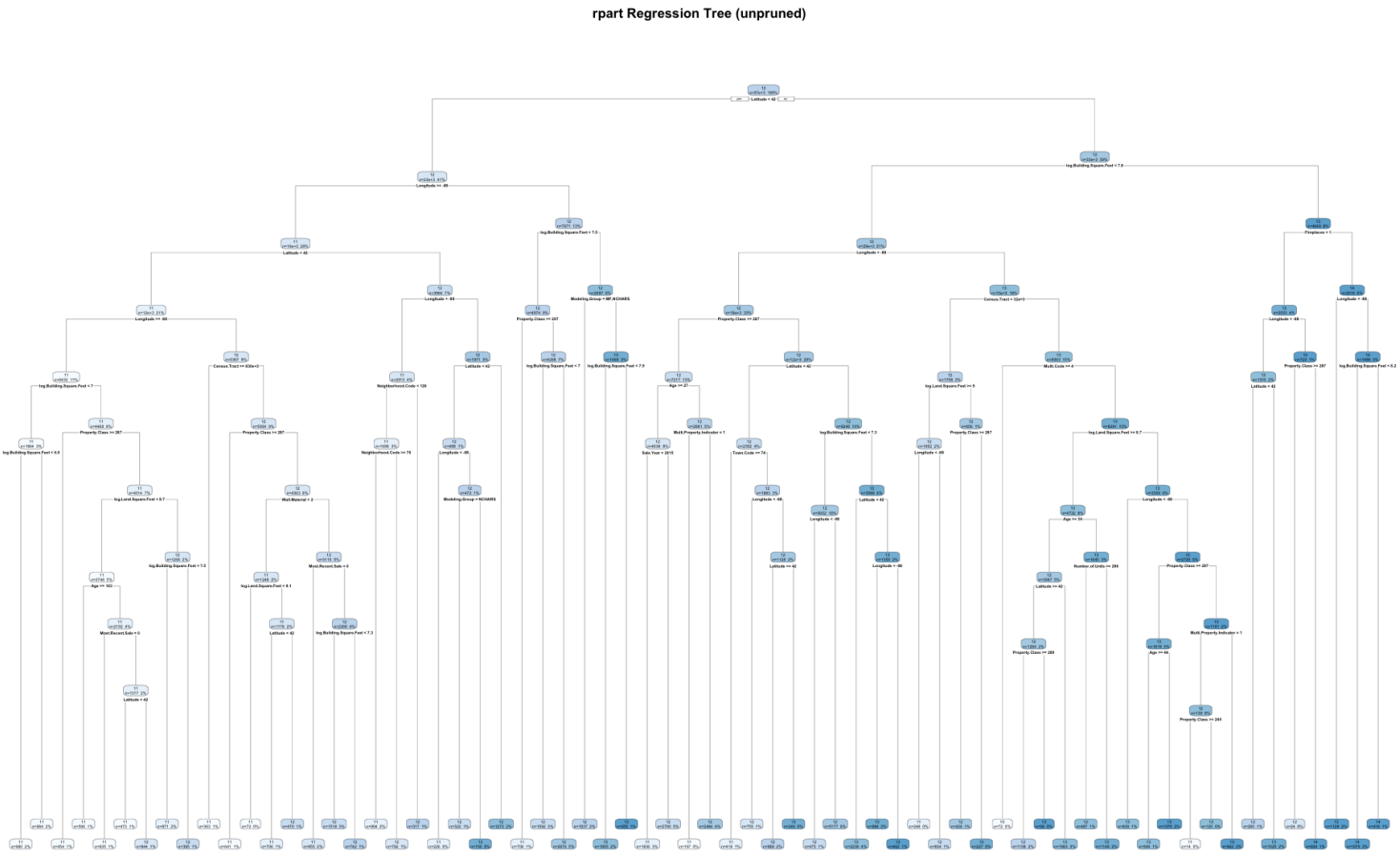


Figure 12: rpart tree

2.6 Decision Tree Regressor (CART in Python)

We fitted `sklearn.tree.DecisionTreeRegressor` with `criterion="squared_error"`, `min_samples_split = 20`, and `ccp_alpha = 0.001`. The same median imputation and one-hot encoding pipeline used for Random Forest and LightGBM was applied to be able to perform direct comparison.

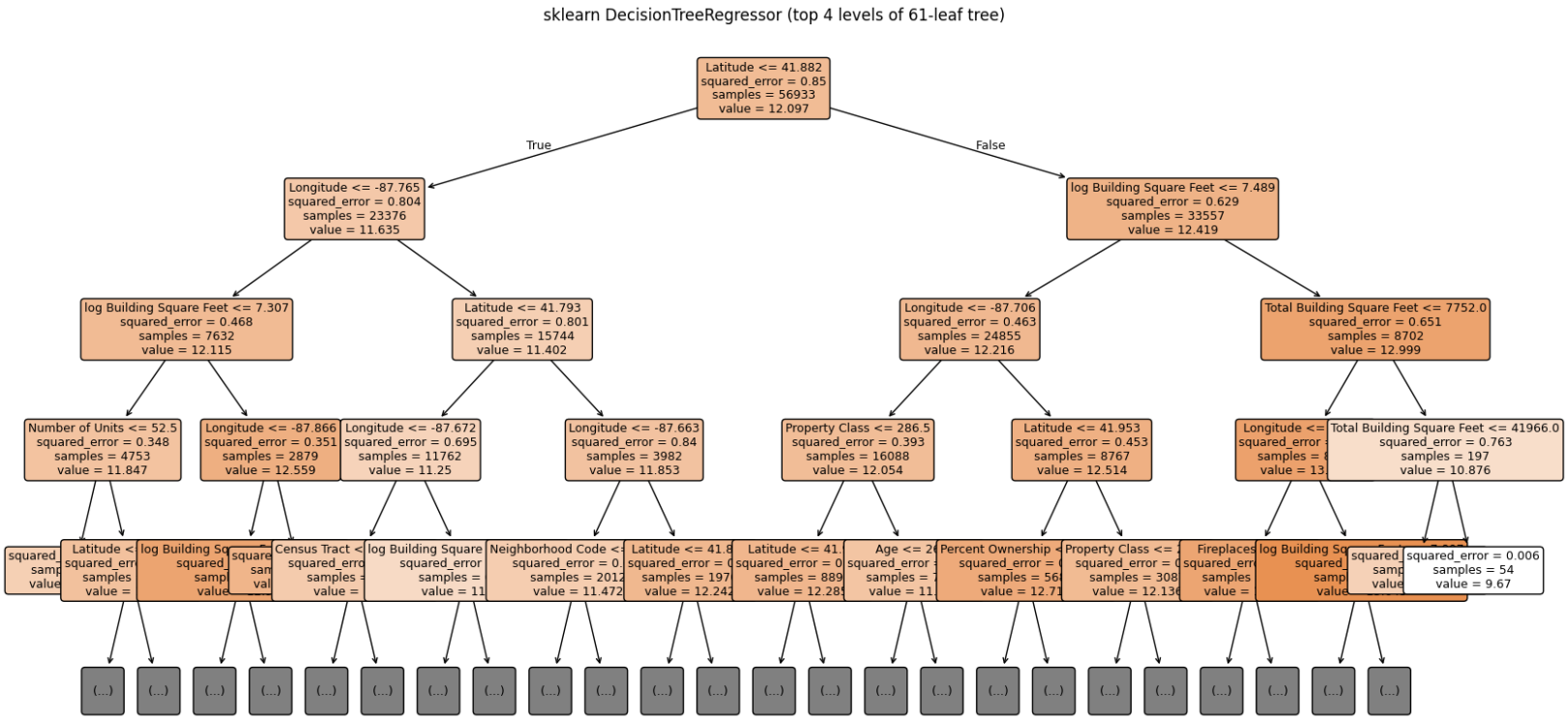


Figure 13: Decision Tree Regressor

Table 11: sklearn DecisionTreeRegressor (CART in Python) – 61 leaves, max depth 10 on 2018–2019 temporal hold-out

Model	**#Leaves**	**Max Depth**	**Log MSE**	**Log RMSE**	**Log R^2**
DecisionTreeRegressor	61	10	0.3167	0.5627	0.5194

2.7 Random Forest

We fitted `sklearn.ensemble.RandomForestRegressor` with `n_estimators = 300`, `max_features = 'sqrt'`, and `min_samples_leaf = 5` (with `oob_score = True`). The same preprocessing pipeline (median imputation + one-hot encoding) was used.

Table 12: sklearn RandomForestRegressor (n_estimators = 300, max_features = 'sqrt') – 2018–2019 temporal hold-out

Model	**Log MSE**	**Log RMSE**	**Log R^2**	**OOB R^2**	**n**
Random Forest	0.1933	0.4396	0.7067	0.7671	23,810

Dollar-scale metrics: RMSE = \$159,430, MAE = \$79,298

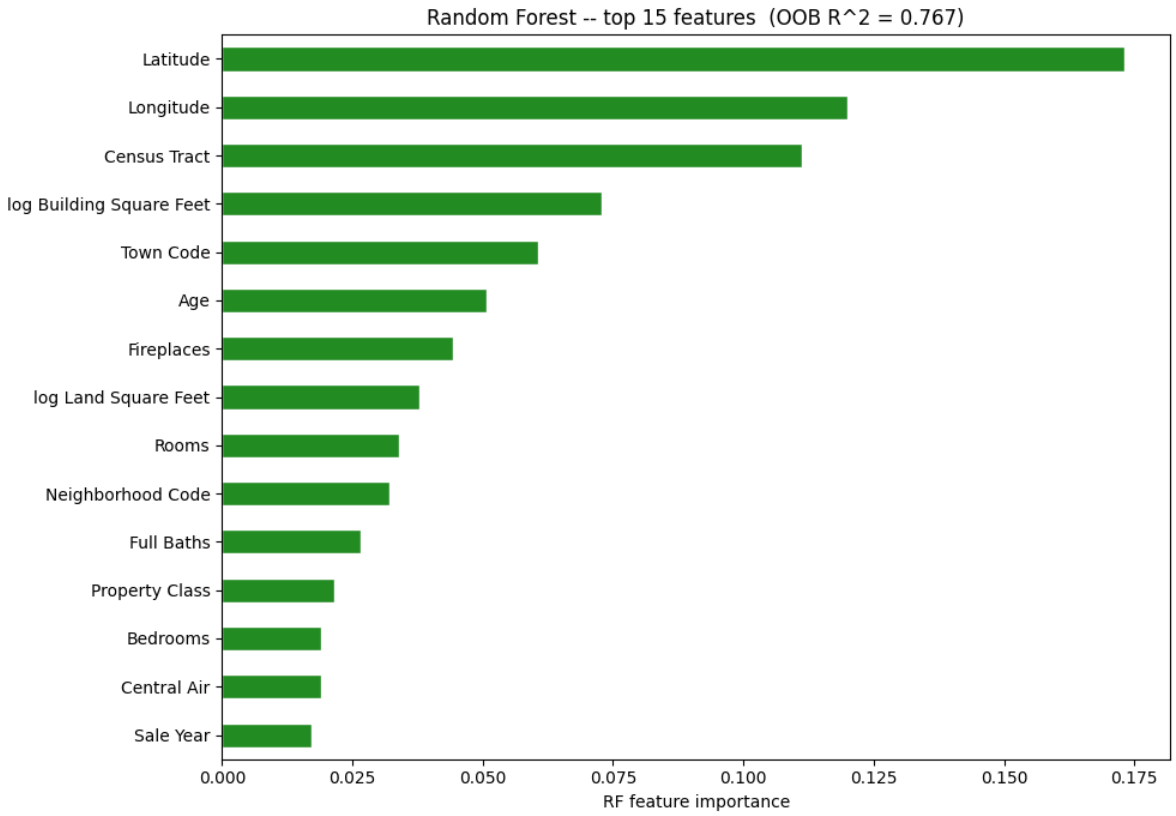


Figure 14: Top 15 features returned by Random Forest (imported from scikitlearn.ensemble)

2.8 LightGBM (gradient boosting)

We trained LightGBM with `objective='regression'`, `metric='rmse'`, `num_leaves=31`, `learning_rate=0.05`, `feature_fraction=0.8`, `bagging_fraction=0.8`, and `min_data_in_leaf=20`. Early stopping with patience = 100 was applied using a 20% validation split of the training data.

Table 13: LightGBM with early stopping (best iteration = 2150) – 2018–2019 temporal hold-out

Model	**Best Iter**	**Log MSE**	**Log RMSE**	**Log R^2**	**n**
LightGBM	2150	0.1570	0.3963	0.7617	23,810

Dollar-scale metrics:

RMSE = \$124,703, MAE = \$64,601

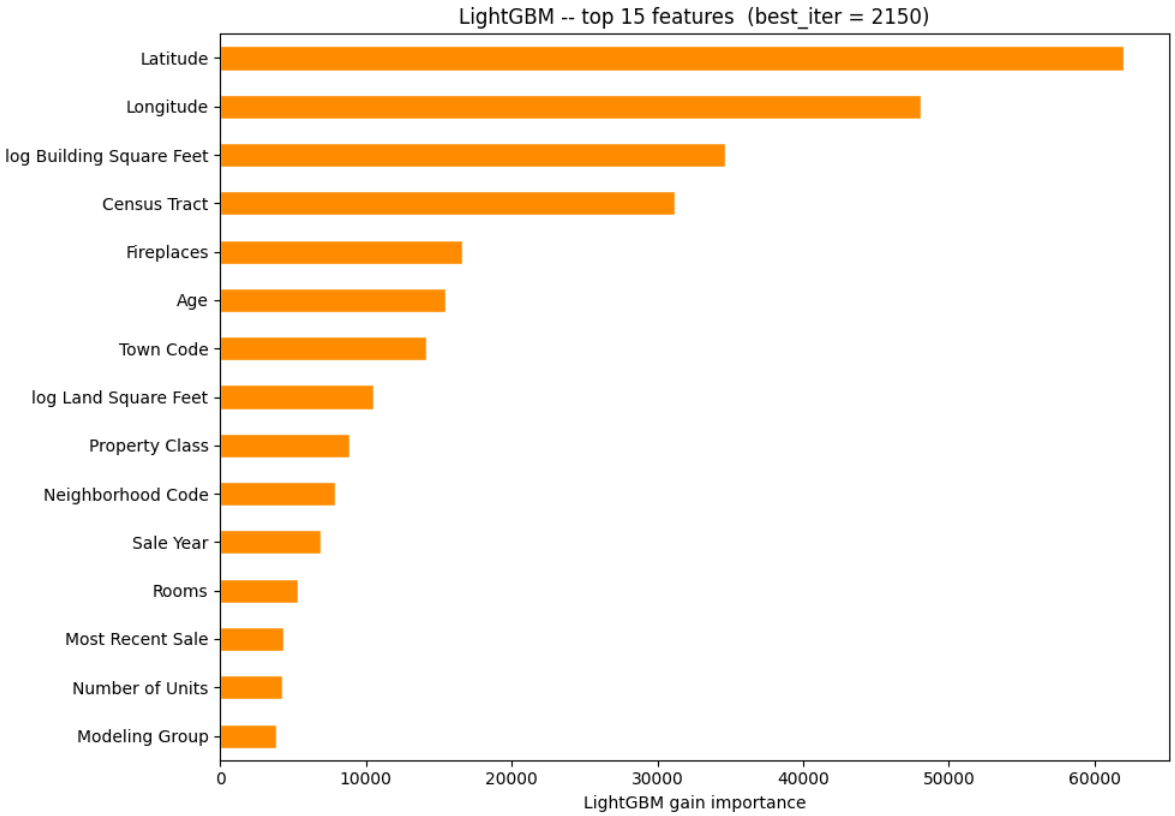


Figure 15: Top 15 features returned by LightGBM

3 Results

The following section presents the final test-set performance on the 2018-2019 temporal hold-out (n = 23,810). We highlight both log-scale metrics (MSE, RMSE, R^2) and, wherever available, dollar-scale metrics. We have drawn key comparisons between different models after the summary table.

3.1 Headline Performance Summary

Key takeaways:

- **LightGBM** achieved the **lowest Log MSE and Log RMSE** (0.1570 / 0.3963) and the best dollar-scale performance (RMSE = \$124,703, MAE = \$64,601).
- **GUIDE Random Forest** attained the **highest Log** R^2 (0.7938), demonstrating that it is able to explain variance well.
- Single-tree models (rpart, sklearn DecisionTree, and even the best GUIDE tree) were clearly outperformed by ensembles, as expected.

3.2 Linear and Ridge Regression Baselines

Linear and Ridge regression ($\alpha = 0.1$) produced nearly identical performance (Log RMSE ≈ 0.595, $R^2 \approx 0.463$). Restricting predictors to only GUIDE-ranked features or applying the `var_to_drop`

Table 14: Test-set performance on the 2018–2019 hold-out (log Sale Price).

Model	**Log MSE**	**Log RMSE**	R^2
Linear regression	0.3537	0.5947	0.4633
Ridge regression ($\alpha = 0.1$)	0.3534	0.5945	0.4637
rpart (CART in R)	0.3265	0.5714	0.5045
sklearn DecisionTreeRegressor	0.3167	0.5627	0.5194
GUIDE, multiple-linear leaves	0.2328	0.4825	0.6468
GUIDE, stepwise-linear leaves (v3)	0.2315	0.4811	0.6487
sklearn RandomForestRegressor	0.1933	0.4396	0.7067
LightGBM (early stopping)	**0.1570**	**0.3963**	0.7617
GUIDE Random Forest	0.1752	0.4186	**0.7938**

redundancy removal procedure (described in Section 2.3) yielded virtually the same numbers. This confirms that linear models cannot capture the complex non-linear interactions present in the data.

3.3 Single-Tree CART Models

Both classical CART implementations (`rpart` in R and `sklearn DecisionTreeRegressor` in Python) produced Log R^2 values in the 0.50-0.52 range, significantly lower than that of the GUIDE trees. This performance gap illustrates the advantage of GUIDE's unbiased variable selection procedure over the CART splitting algorithm that favors continuous variables, like latitude and longitude in this case, with many split points.

3.4 GUIDE Regression Trees

GUIDE regression trees substantially outperformed classical CART. The four controlled comparisons (already shown in Section 2.4) revealed several important insights:

- Stepwise-linear leaves slightly outperformed multiple-linear leaves on the temporal hold-out.
- The v3 feature set (raw square footage) outperformed the v2 price-per-square-foot features, confirming leakage of train set sale price data into the test set.
- The temporal hold-out was more reliable than 10-fold CV for final evaluation.

The best single GUIDE tree (stepwise-linear leaves, v3 features (with temporal hold-out)) achieved Log MSE = 0.2315, Log RMSE = 0.4811, and Log $R^2 = 0.6487$.

3.5 Ensemble Methods

Ensemble models delivered the strongest overall results. The GUIDE Random Forest is able to explain the variance the best ($R^2 = 0.7938$). LightGBM, with early stopping at iteration 2150, produced the most accurate dollar-scale predictions (RMSE = \$124,703, MAE = \$64,601). These results demonstrate that while GUIDE provides excellent interpretability and unbiased importance scores, modern gradient boosting algorithms are able to achieve higher predictive accuracy by combining many weak learners.

3.6 Goals for Future Improvement

Goals for future improvement: If time permits, I would add binary columns for features that have high percentage of missingness to make it a stronger indicator for signal. This is especially useful for datasets like this with huge degrees of structural missingness.